

Past, Present, and Future of Bug Tracking in the Generative AI Era

UTKU BORAN TORUN, Bilkent University, Department of Computer Engineering, Türkiye

MEHMET TAHA DEMIRCAN, Bilkent University, Department of Computer Engineering, Türkiye

MAHMUT FURKAN GÖN, Bilkent University, Department of Computer Engineering, Türkiye

ERAY TÜZÜN, Bilkent University, Department of Computer Engineering, Türkiye

Traditional bug tracking systems rely heavily on manual reporting, reproduction, classification, and resolution. These tasks are performed by various stakeholders, including end users, customer support, developers, and testers. This division of responsibilities requires significant coordination in addition to the considerable human time and effort. This reliance widens the communication gap between non-technical end users and developers, who are more technically inclined, slowing the process from bug discovery to resolution and deployment. Moreover, current solutions tend to be highly asynchronous; end users often wait hours, days, or even weeks before receiving an initial response, further delaying fixes and contributing to frustration.

In this study, we examine the evolution of bug tracking practices, moving from early paper-based and manual reporting methods to today's web-based platforms that dominate modern software development. Building on this trajectory, we outline our vision for the future: an AI-powered bug tracking framework that augments existing systems with intelligent, large language models (LLMs) and agent-driven automation, and we report on early adaptations of key components of this framework that provide initial empirical grounding for its feasibility.

Our approach tackles two main challenges: reducing the time to resolution and minimizing coordination overhead by bridging the gap between end-users and developers. In the proposed framework, end users report bugs in natural language. AI-driven agents refine these reports, attempt to reproduce them, and request any missing details. Bug reports are classified according to their severity, priority, and type. Valid and invalid bugs are automatically determined, and the invalid bug reports are either dismissed or resolved via no-code fixes suggested by agents. Valid bugs, once confirmed, are localized and assigned to developers, who review LLM-generated patches and verify their correctness. Verified patches are deployed using continuous integration and continuous development (CI/CD) pipelines.

This paper articulates our vision for an AI-powered bug tracking framework and examines the challenges and opportunities of integrating LLMs into bug tracking workflows. We position the framework as a forward-looking design that can also augment today's issue trackers, illustrating how automation can reduce turnaround time while reshaping software maintenance into a more efficient, collaborative, and user-centric process.

CCS Concepts: • **Software and its engineering** → **Maintaining software**; • **Artificial intelligence** → *Intelligent agents*.

Additional Key Words and Phrases: Bug Tracking, Large Language Models, AI-Driven Software Maintenance, AI-Powered Software Engineering, Bug Reports, AI-Powered Bug Tracking

Authors' addresses: Utku Boran Torun, boran.torun@bilkent.edu.tr, Bilkent University, Department of Computer Engineering, Ankara, Türkiye; Mehmet Taha Demircan, taha.demircan@bilkent.edu.tr, Bilkent University, Department of Computer Engineering, Ankara, Türkiye; Mahmut Furkan Gön, furkan.gon@bilkent.edu.tr, Bilkent University, Department of Computer Engineering, Ankara, Türkiye; Eray Tüzün, eraytuzun@cs.bilkent.edu.tr, Bilkent University, Department of Computer Engineering, Ankara, Türkiye.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/4-ART

<https://doi.org/10.1145/3806655>

1 Introduction

Bug tracking is a fundamental process in software maintenance, ensuring software quality, reliability, and long-term sustainability [79]. The concept of software bugs dates back to the early days of computing, with the famous case of a moth causing a malfunction in the Harvard Mark II computer in 1947, often cited as the first recorded bug [122]. Since then, the occurrence of defects in software systems has been recognized as an inevitable aspect of development, prompting the establishment of systematic practices to identify, document, and resolve issues. Early approaches to bug tracking were predominantly manual, involving handwritten logs, spreadsheets, or informal communication among developers [64]. As software engineering matured, the growing scale and complexity of systems required more structured methods, leading to the development of dedicated bug tracking systems in the late 20th century [107]. These systems introduced standardized reporting formats, status tracking, and mechanisms for assigning responsibility, creating a more disciplined process for bug tracking.

Over time, bug tracking has evolved into an essential backbone of collaborative software development, becoming integrated into broader software engineering workflows. Modern bug tracking practices emphasize traceability and coordination among diverse stakeholders [9], enabling teams to track the entire lifecycle of a bug, from discovery and reporting to resolution and verification. Bug tracking repositories now serve not only as records of defects but also as valuable sources of knowledge, informing project management decisions, supporting quality assurance, and guiding future development [181]. Despite these advancements, the fundamental manual nature of bug tracking has remained largely unchanged. Consequently, the process is hindered by communication gaps, ambiguous reports, and high reliance on human intervention [8].

The correlation between increasing project scale and bug frequency underscores the necessity of robust bug tracking systems as a cornerstone of software quality assurance [57, 58]. Widely used bug tracking tools such as Jira¹, Bugzilla², and GitHub Issues³ have played central roles in software engineering. Jira, released in 2002, has become a dominant system in enterprise settings for handling bugs, tasks, and project management workflows. Bugzilla, publicly available since 1998, has been used in many open source projects as one of the canonical issue tracking platforms, particularly for its flexibility and openness. GitHub Issues has also emerged as one of the most widely used bug tracking systems in open source communities. These tools are used widely across open source communities and proprietary organizations, often acting as the canonical system for bug reporting, triage, assignment, status tracking, historical analysis, and the communication channel for the stakeholders.

However, despite their maturity and widespread adoption, these systems have notable deficiencies. These systems might be thought of as serving as an interface for database operations. Moreover, Jira and Bugzilla often impose overhead by requiring detailed workflow configuration and many mandatory fields, which can slow down bug reporting and triage. For example, Jira users have complained about performance and usability issues, like the system slowing down when workflows get too complex⁴. In contrast, GitHub Issues has been criticized for its unstructured design. Empirical studies show that many issues are created without sufficient context, lacking reproducible steps or environment details [120]. Software maintainers often cannot, or choose not to, enforce strict templates or workflows in many repositories. For example, a study on large GitHub projects [131] found that the schema of issue templates and required metadata is either minimal or inconsistently used, leading to ambiguity in many reports. These deficiencies result in several inefficiencies, including duplicate reports, slower triage and resolution, inconsistent issue quality, and higher manual efforts.

The recent rise of large language models (LLMs) such as Gemini [33], GPT [2], and Llama [135] has transformed natural language processing (NLP) by enabling machines to generate, understand, and reason over text at

¹<https://www.atlassian.com/software/jira>

²<https://www.bugzilla.org/>

³<https://github.com/features/issues>

⁴<https://jira.atlassian.com/browse/JRASERVER-61704>

unprecedented levels of fluency and accuracy. Leveraging massive pre-training on diverse text corpora, these models demonstrate remarkable zero-shot and few-shot capabilities, generalizing across a wide variety of tasks without requiring task-specific training. Beyond research benchmarks, LLMs are increasingly deployed in real-world applications such as conversational assistants, code generation, and knowledge management, fundamentally changing the way humans interact with software systems. Their ability to interpret unstructured inputs, extract structured information, and produce contextually relevant outputs suggests strong potential for integration into complex workflows that usually exist in software engineering. GitHub Copilot⁵, for instance, has already shown how LLMs can be used in programming tasks by providing real-time code suggestions, signaling a shift toward AI-augmented software development [138].

Beyond standalone LLMs, researchers are increasingly focusing on LLM agents, autonomous or semi-autonomous systems that can understand their environment, plan tasks, and carry out actions to achieve long-term goals [147]. These agents can be combined into multi-agent systems, where multiple LLM agents collaborate through structured communication, task specialization, and coordination protocols [63, 111]. Such orchestration allows agents to complement each other, mitigate individual weaknesses, and scale to solve more complex problems than a single model could handle in isolation. Frameworks such as AutoGPT⁶ and MetaGPT⁷ exemplify this direction, where agents assume distinct roles (e.g., planner, coder, tester) and cooperate to achieve complex objectives. The emergence of these paradigms highlights the feasibility of constructing collaborative ecosystems of LLMs that can continuously refine, validate, and operationalize knowledge in dynamic domains.

Integrating LLMs into bug tracking is underexplored but vital, addressing the communication issues⁸, resolution delays⁹, and manual overhead¹⁰ typical of traditional methods. While recent works have applied LLMs to some tasks in bug tracking, such as bug localization [102, 157] and automated program repair [110, 180], others have explored their use in broader software engineering activities [59, 65, 112]. However, the integration of LLM agents into the whole end-to-end bug tracking lifecycle has not been systematically studied. Moreover, the potential of collaborative LLM agents to act as automated triagers, verifiers, or explainers of bug reports has yet to be fully realized. This gap presents a promising research opportunity to reimagine bug tracking systems by embedding LLMs and multi-agent collaboration into their core, thereby reducing manual effort, improving report quality, and accelerating defect resolution.

In this article, we introduce a visionary AI-powered framework that leverages LLMs to automate key aspects of the bug tracking process. Our framework enables users to report bugs in natural language, allowing AI-driven automation to assist in reproduction, classification, validation, resolution, and verification. By reducing reliance on manual triaging and streamlining communication, our approach would enhance efficiency while maintaining human oversight where necessary, and decrease time to resolution (TTR).

By automating repetitive and time-consuming tasks and bridging the gap between end-users, who are usually non-technical, and developers, who form the technical side, our vision aims to reduce the time and human effort required for bug reporting and resolution. It also addresses communication issues¹¹ that often arise during bug tracking. Through automation, we aim to minimize human involvement in repetitive tasks¹² that typically happen during bug tracking while maintaining high accuracy and efficiency in bug resolution. We discuss the potential challenges and long-term research opportunities in integrating AI-driven automation into software

⁵<https://github.com/features/copilot>

⁶<https://github.com/Significant-Gravitas/AutoGPT>

⁷<https://github.com/FoundationAgents/MetaGPT>

⁸<https://github.com/microsoft/vscode/issues/519>

⁹<https://github.com/kubernetes/kubernetes/issues/117706>

¹⁰<https://github.com/pytorch/pytorch/issues/77764>

¹¹<https://github.com/microsoft/vscode/issues/519>

¹²<https://github.com/pytorch/pytorch/issues/77764>

maintenance workflows, specifically in bug tracking. We envision a future where AI and human collaboration, within a human-in-the-loop (HIL) system, redefines software maintenance.

In the following sections, we first explain the historical evolution of bug tracking systems from early solutions to the contemporary bug tracking systems with their limitations in Section 2, our proposed bug tracking framework in Section 3, then, we discuss the possible implications and open challenges in Section 4. We conclude our study in Section 5.

2 Historical Evolution and Traditional System of Bug Tracking

This section provides an analysis of the bug tracking process, from its foundational principles to its modern implementations. Section 2.1 reviews the evolution of bug tracking systems, showing how early manual approaches provided the foundation for today's advanced platforms. Section 2.2 describes the workflow of the traditional bug tracking systems, detailing their key components and how they function within the software development lifecycle. Finally, Section 2.3 critically evaluates the challenges of these traditional systems, identifying the challenges and inefficiencies.

2.1 Historical Evolution of Bug Tracking Systems

The evolution of bug tracking systems reflects the broader maturation of software engineering practices, moving from informal, ad hoc issue recording methods to sophisticated, collaborative platforms integrated across the software development lifecycle. Understanding this trajectory is crucial to identifying current limitations and informing the design of next generation systems. In the following, we explain bug tracking eras, which are divided according to their key technological aspects and innovations. Table 1 displays each era and its characteristics.

2.1.1 Early Digital Era (1940s-1970s). In the earliest days of software development, bug tracking was a manual, often paper-based process. Developers recorded errors in logbooks or simple text files. These methods lacked structure and traceability, offering no systematic way to categorize, prioritize, or assign bugs. For instance, during the operation of ENIAC [128], the most common errors were notational, arising from inconsistencies among redundant specifications of the same detail. Also, there were concurrency errors, which were resolved by adjusting the timing of control signals, often by inserting a dummy program to delay a signal. As projects grew in scale and complexity, manual tracking became untenable [114].

2.1.2 Pre-Internet Era (1970s-1980s). During this period, communication began to evolve beyond face-to-face meetings. Email systems^{13,14}, simple databases like dBase¹⁵, and early file-sharing networks emerged. Bugs were often reported by users to customer support via phone calls, and with early email communication relying on private or institution-specific networks before widespread internet access became common. The software itself was usually shipped on floppy disks, and users would sometimes mail back faulty disks along with registration forms. When a customer support representative received a bug report, they would write the reproduction steps into a text file. This file would then be passed to a developer who would manually try to reproduce the bug. Fixing a bug involved manual code changes, which were then shared with the customer via a new floppy disk. Verification was done through manual testing with a documented checklist. The process was very slow and inefficient. While remote communication was a new development, collaboration was still low, often leading to a high TTR.

¹³<https://en.wikipedia.org/wiki/ALL-IN-1>

¹⁴https://en.wikipedia.org/wiki/IBM_OfficeVision

¹⁵<https://www.dbase.com>

2.1.3 Internet Era (1980s-1990s). This era saw the first dedicated bug tracking systems. Tools like GNATS (GNU Bug Tracking System)¹⁶, Debbugs¹⁷, and CMVC [172] were introduced. GNATS, for instance, was an early, open source bug tracking system that used text-based files and email for communication, providing a structured interface for logging and tracking bugs. Debbugs was the software powering the Debian project's issue tracking system. It did not have any form of web interface to edit bug reports, all modification was done through email. These systems allowed for detailed reproduction steps to be logged and were accessible to the whole team. When a bug was fixed, the code was committed to a version control system (VCS), and the bug report was updated with details about the fix.

As software development matured, teams began to adopt formal processes. Developers and quality assurance (QA) engineers tracked basic bug statistics such as the number of new bugs opened, bug close rates, and fix versus invalid rates.

To manage the bug backlog, Netscape¹⁸ team leaders held weekly bug meetings, or "bug councils" [35]. As the project neared its release, these meetings increased in frequency to once or twice a day to prioritize and make decisions on fixes. These meetings were typically led by a project or release manager and included representatives from development, QA, and marketing teams. Netscape used a five-level classification system; critical, major, normal, minor, and trivial. Bugs were classified and entered into the database by the engineers who found them, typically from the QA team. The marketing team also maintained a "top 10 bugs" list based on customer feedback, which was passed to QA.

2.1.4 Web-Based Era (2000s). This decade saw a major shift to web-based platforms. Tools like Bugzilla, MantisBT¹⁹, Trac²⁰, and early versions of Jira emerged. Bugzilla became a landmark open-source system. Bugzilla introduced foundational abstractions like structured fields (e.g., severity, priority, component), status transitions, and user roles. It provided a web-based interface and notification mechanisms, enabling better coordination among developers and testers [108]. While Bugzilla established the foundational blueprint for web-based tracking, Jira eventually surpassed it by offering comprehensive project management features and a flexible architecture that became the dominant choice for enterprise and agile teams. These systems treated bug reports as work items in a sprint, integrating with the emerging agile methodologies. The web UI made it much easier for everyone to access and track bugs. The expansion of community-contributed plug-ins and themes also reflected the diverse needs of development teams, from small startups to large open source foundations [18]. Collaboration improved with shared dashboards and email notifications, though bottlenecks were still common due to a lack of deep integration with development tools [42].

2.1.5 DevOps and Automation Era (2010s-2022). In this era, platforms like GitHub Issues, GitLab Issues²¹, YouTrack²², and Azure DevOps²³ have emerged as popular bug tracking systems. Bug tracking became fully integrated into the development lifecycle. GitHub Issues, for example, is part of a larger ecosystem that includes version control and continuous integration. Fixes were no longer just committed; they became a part of a Continuous Integration/Continuous Deployment (CI/CD) pipeline, where code changes automatically trigger tests and deployments.

¹⁶<https://www.gnu.org/software/gnats/>

¹⁷<https://en.wikipedia.org/wiki/Debbugs>

¹⁸<https://isp.netscape.com/>

¹⁹<https://mantisbt.org/>

²⁰<https://trac.edgewall.org/>

²¹<https://docs.gitlab.com/user/project/issues/>

²²<https://www.jetbrains.com/youtrack/>

²³<https://azure.microsoft.com/en-us/products/devops>

The growing popularity of issue-driven development, where code commits are directly tied to issue identifiers, further solidified the role of bug tracking systems as central elements of the software delivery process. Developers began to view bug tracking systems not merely as databases or interfaces for bug database operations, but as vital components of planning, communication, and accountability [22, 52].

Parallel to industry adoption, academic research began to explore how Machine Learning (ML) techniques could improve the efficiency of bug tracking processes [113]. Early studies applied text classification and clustering algorithms to automate tasks such as duplicate detection, severity prediction, and assignee recommendation [27, 89]. These contributions laid the groundwork for more intelligent and data-driven approaches to defect management.

Table 1. Summary of Bug Tracking Eras

Era	Bug Reporting	Bug Reproduction	Bug Fixing	Verification	Bug Classification Support	Representative Tools	What is New?
Early Digital Era (1940s-1970s)	Bugs were written to the notebooks, sticky notes, spreadsheets, etc.	There are a certain number of testing methods in the system, and with these testing methods, the bug is reproduced.	For notational bugs, bugs are fixed by correcting the wrong place in the notebook. For the concurrency bugs, the bugs are fixed by using a dummy program to adjust the signals.	The same methodology as the Bug Reproduction. If all tests are passed, then the bug is verified.	Bug types or severity are not consistently captured	Paper Logs, Index Cards, Notebooks	Problems that may occur in hardware and software solutions should be addressed.
Pre-Internet Era (1970s-1980s)	Reported via email/phone or customer support. Software shipped on floppy disks with registration forms; users mailed forms and faulty disks back to developers.	Customer support receives bug reproduction steps from the user via email or phone call, then writes these steps to the text file. Then these steps are passed to the developer to reproduce the bug.	Manual code changes, shared via floppy disk or a local network.	Manual testing, often with a documented checklist.	Simple severity labels, often inconsistently used	Email, text editors, simple databases (e.g., dBase), network file shares, Lotus 1-2-3. ²⁴	Introduction of remote communication for bug reporting (email, phone) and use of digital storage for transferring software and fixes.
Internet Era (1980s-1990s)	Reports were logged into the system by a user with some predefined structure.	Detailed Steps to Reproduce (S2R) were logged in the system, accessible to the whole team.	Code fixed and committed to VCS. Updates were often manual in the bug report.	Formal QA teams re-tested fixes; verification was largely manual and sequential.	Simple severity labels, often inconsistently used	GNATS, Debuggs, CMVC	Structured digital bug databases introduced, enabling team-wide visibility and linking bugs to code changes.
Web-Based Era (2000s)	Web UI support, bug reports were treated as work items in a sprint	Web UI S2R were logged in the system, accessible to the whole team.	Fixes tied directly to modern VCSs. Systems started auto-linking commits/PRs to bug reports.	QA remained formalized but incorporated automation (test scripts, regression suites) and parallel workflows.	Manual categorization with some automation	Bugzilla, MantisBT, Trac, Redmine, Jira, Microsoft Team Foundation Server (TFS)	Shift to web-based collaboration, integration with Agile workflows, and more formalized bug management processes.
DevOps and Automation Era (2010s-2022)	Web-based, collaborative platforms integrated with other development tools.	Logs/screenshots attached to tickets; testing environment integration became standard	Fixes were part of a CI/CD pipeline. Fixes were tracked via VCSs.	Automated testing became a standard practice, with test results often linked directly to the bug ticket.	Tagging, priority queues, category rules	GitHub Issues, GitLab Issues, YouTrack, Azure DevOps, Jira	Fully integrated bug tracking into DevOps toolchains with CI/CD, automated testing, and real-time collaboration.

2.2 Traditional System

Figure 1 presents a typical workflow of the traditional bug tracking system in DevOps, and Automation Era, showing the interactions among different roles. The process starts when a user creates a bug report. Customer support then attempts manual bug reproduction. If reproduction is unsuccessful, additional details are requested from the user, and the attempt is repeated until the bug can be reliably reproduced. Once reproduction is successful, the bug proceeds to classification. After classification, a bug-feature traceability step is performed, followed by the validity check. In the traditional literature, classification and assignment together are often referred to as “bug triage,” but in our discussion, we separate these into two distinct stages: classification and assignment.

If the bug is invalid, customer support attempts what we call a no-code fix, meaning a resolution handled outside the codebase, for example, updating documentation, adjusting configurations, or clarifying usage. This resolution is then verified by customer support, and if verification succeeds, the user is directly notified and asked to confirm whether the resolution is acceptable, which we call user verification. If customer support verification fails, the process returns to the no-code fix stage, where customer support attempts another resolution until the issue can be successfully verified.

²⁴https://en.wikipedia.org/wiki/Lotus_1-2-3

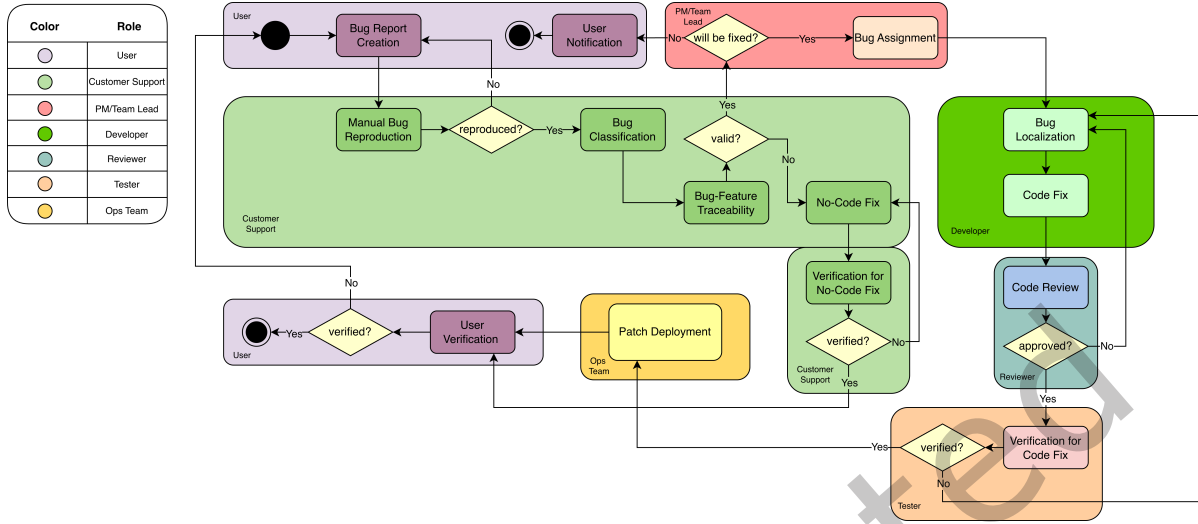


Fig. 1. Overview of Traditional Bug Tracking System Workflow

For valid bugs, the Project Manager (PM) or team lead determines whether the issue should be addressed. A negative decision results in the user being notified, while a positive decision leads to the bug being assigned to a developer. The developer first performs bug localization and then applies a code fix. After the fix, the change enters the code review stage. If the reviewer does not approve the fix, it cycles back to the developer; if it is approved, the fix moves to the verification stage.

The tester verifies whether the code fix has resolved the bug. If the verification fails, the process returns to the developer; if it succeeds, the workflow continues with patch deployment carried out by the operations team. Once the patch is deployed, user verification serves as the final step. If the user confirms the fix, the process ends successfully; if not, the workflow loops back for further handling.

2.3 Challenges of Traditional System

Given the traditional bug tracking process described above, it is important to examine the key challenges that hinder its effectiveness. While the primary objective is to identify and resolve bugs within an intended time span, the whole process, which starts with bug reporting and ends with final deployment and user verification, faces many obstacles. These challenges are not solely technical; they are also shaped by human, organizational, and workflow-related factors. In particular, prior studies have shown that recurring bug tracking process smells, that is, deviations from recommended bug tracking practices, can negatively affect workflow efficiency and software quality [7, 115, 137, 181]. In the following, we summarize the key challenge across the major phases of the bug tracking lifecycle.

2.3.1 Challenges of Initial Bug Report Creation. The bug-fixing process is fundamentally tied to the quality of the information provided in the bug report, which is one of the cornerstones of a bug tracking system [181]. A primary challenge is that information in these reports is often incomplete or vague [24, 132]. This can lead to reports that lack clear reproduction steps and crucial environmental details. As a result of these bad practices, customer support staff and developers are forced to engage in a time-consuming "ping-pong" of communication with the reporter, requesting additional information [115]. This back-and-forth exchange significantly slows down the bug resolution process and increases the number of unresolved issues [182]. The problem is often

compounded by the passive nature of traditional bug tracking systems, which are little more than interfaces for relational databases and offer minimal support for reporters to provide the necessary context [182]. In addition, reporters who are not technically experienced may struggle to articulate problems in a structured way or find bug tracking interfaces difficult to use, further reducing the clarity and usefulness of bug reports.

2.3.2 Challenges of Bug Reproduction. Once a bug is reported, the next step is consistently reproducing it in a controlled environment. This task can be rather difficult due to several reasons, such as discrepancies between the original user environment and the development environment or lack of information in the bug report [95, 140]. Software defects that manifest under non-deterministic conditions significantly complicate the process of reproducibility. A classic example is the 'Heisenbug' [54], a term describing bugs that alter their behavior or disappear entirely when being observed or debugged. Such defects, often exemplified by concurrency issues, remain notoriously difficult to systematically replicate [66].

The rate of non-reproducible bugs is far from negligible; in their study, Goyal et al. [53] mined bug reports from four large-scale open source software projects of the Bugzilla repository and found that non-reproducible bugs range from 12.77% to 24.26% of all reported bugs. Failing to reliably reproduce an issue often leads to frustration and wasted effort for both the development team and users.

2.3.3 Challenges of Bug Classification. Manual classification is labor-intensive and time-consuming, which involves a series of sequential actions, demands substantial problem-solving abilities, and careful consideration. The large volume of incoming bug reports can easily overwhelm teams, leading to a mounting backlog of issues and significant delays in the software development lifecycle [84]. The prevalence of incomplete and ambiguous descriptions in bug reports poses a significant challenge to accurate classification [20]. For instance, Zhang et al. [174] found that 78.1% of Mozilla and Eclipse reports consist of fewer than 100 words, creating a lack of information that prolongs the resolution process.

Another persistent challenge is subjectivity, particularly in the critical steps of assigning severity and priority. Different team members may interpret the same symptoms differently, causing inconsistencies in triage decisions. To mitigate this, many teams adopt structured frameworks or agreed-upon taxonomies to align technical and business assessments of a bug, which is hard to maintain.

2.3.4 Challenges of Traceability of Bugs to Other Artifacts. Traceability of a bug refers to the ability to track it throughout its entire lifecycle using artifact links. Examples for these artifacts can be bug-fixing commits [118] and pull requests (PRs) of the bug fix [168]. Due to a lack of set rules and oversight, developers often fail to link their PRs to the specific issues they are meant to solve [119]. This disconnect makes it difficult to understand why certain code was changed, resulting in the loss of valuable information that could be useful when problems arise. Another important aspect is Requirements Traceability (RT), which refers to the ability to track and follow the life of a requirement across the project lifecycle [51]. RT links are valuable during maintenance because they allow developers to quickly understand the impact of changes, trace defects back to their originating requirements, and ensure that modifications remain consistent with stakeholder needs. However, creating and maintaining these links is laborious, as it often requires manual effort to analyze requirements, design artifacts, code, and test cases, and then establish explicit relationships between them. Lack of traceability can raise problems in the future of that software project, such as the reduced effect of change impact analysis [15]. Automated techniques to construct these links would save resources.

2.3.5 Challenges of Detecting Invalid Bugs. Another possible challenge is to detect if the bug is valid or not. Some of the causes of invalid bugs are insufficient background knowledge of the reporter, misunderstanding of functionality, misunderstanding of environment, error in testing, and error in the external system, according to Sun et al. [132].

While automated detection of invalid bug reports using ML has shown promise in reducing manual effort, its adoption faces several practical challenges. A major issue is the quality and imbalance of real-world bug datasets. Projects often contain a large proportion of invalid or duplicate reports compared to valid bug reports, which biases models toward the majority classes and reduces performance on the more critical minority classes [134]. Beyond imbalance, bug reports themselves are highly heterogeneous: they may be written in natural language with varying levels of detail, include incomplete or noisy information, or use project-specific jargon that makes feature extraction difficult. Additionally, labeling ground truth data is labor-intensive and prone to human inconsistency, further limiting the reliability of training data. These challenges collectively highlight why practical deployment of automated bug validity determination remains non-trivial despite promising research results.

2.3.6 Challenges of Bug Localization. A primary challenge in debugging is the initial task of identifying the root cause of an issue. This task is particularly demanding in complex codebases, where developers must isolate the root cause from a vast number of potential sources [49]. The difficulty is compounded in modern environments by the distributed and interconnected nature of applications, where issues may traverse multiple system layers and require expertise with a range of specialized tools [49].

2.3.7 Challenges of Creating a Bug Fix. Addressing invalid bug reports, which might stem from misconfigurations, incompatible data usage, or simple permission issues rather than actual code defects, presents a significant workflow bottleneck. Currently, the resolution of these issues relies heavily on manual intervention by customer support staff, who must diagnose the environmental mismatch and apply no-code fixes such as directing users to make changes in their configuration files or user access levels. This dependency on manual intervention creates substantial latency; even when the solution is as simple as a permission update from the user side, the asynchronous nature of communication for support often forces end-users to wait hours or even days for a resolution. Moreover, because these operational changes occur outside the rigorous CI/CD pipelines used for code patches, they are prone to human error and lack verification and automation.

Regarding valid bug reports, while a bug might be identified and its root cause understood, the actual process of changing the code is far from trivial. Typically, a change made to a software system may result in an undesirable side effect to the rest of the system [12]. There is a unique set of challenges that can impact code quality, introduce new defects and code smells, and increase the overall development cost. Changes to the code might affect other parts of the code, which can be challenging to detect, as change impact analysis is used to measure the side effects of code changes using several metrics [50]. A lack of documentation may also cause challenges in the code change process. Without clear documentation, developers may struggle to understand why a particular piece of code was written in a certain way. Furthermore, the nature of the fault itself can render the fixing process exceptionally difficult. Specifically, challenges arise from Mandelbugs [55] and their subtype, Heisenbugs. While Mandelbugs are characterized by complex, non-deterministic activation mechanisms, Heisenbugs specifically exhibit behavior that shifts under observation or during debugging. These transient issues, often rooted in race conditions or timing delays, complicate the classification process because they are inherently difficult to isolate, reproduce, and verify.

Beyond the complexity of the fault logic, many failures cannot be fixed through source code modifications or no-code fixes alone. This category of defects originates from the state of the execution environment rather than internal algorithmic errors. For example, a failure might be triggered by resource exhaustion, like a memory leak in a third-party library, or inconsistent schema states between shared databases. In such instances, a standard source-level patch is insufficient; the resolution instead requires environmental change, such as reconfiguring container resource limits.

2.3.8 Challenges of Bug Fix Verification. Verifying a bug fix is the final, critical step in the bug-fixing lifecycle. It is the process of confirming that the bug is truly resolved and that the fix has not introduced any new problems. The primary technical challenge in bug fix verification is the risk of regression. A bug fix, especially in a complex system, can inadvertently break existing, previously functional code [171]. This requires testers to go beyond merely checking if the original bug is fixed and perform extensive regression testing, which can be time-consuming and difficult without robust automated test suites. Another technical challenge in this step is reproducibility, which mirrors the difficulties discussed in Section 2.3.2. Specifically, this is closely linked to the challenge of flaky tests, tests that fail intermittently without any code changes [101]. The scope of testing is also a challenge, as testers must correctly identify all areas of the application that could have been affected by the fix, a task that becomes exponentially more difficult with tightly coupled code and a lack of clear documentation [12]. Beyond the technical aspects, bug fix verification is significantly affected by human and organizational factors. Deak et al. [36] demonstrate that testers are affected by negative factors, including a lack of influence and recognition, dissatisfaction with management, technical issues, inadequate organization, time pressure, boredom, poor relationships with developers, and issues with the working environment.

2.3.9 Challenges of Asynchronous Communication. In modern bug tracking workflows, much of the interaction between stakeholders takes place through asynchronous communication, that is, exchanges like emails, issue comments, or chat messages that do not require participants to be online at the same time. The primary challenge of using asynchronous communication is the inherent lack of real-time context and immediate feedback. Unlike synchronous methods such as live meetings or screen shares, asynchronous bug reporting and resolution can lead to significant delays [42]. A developer may need to request more information about the bug, such as specific S2R for it or screenshots from a different browser, but the response may not arrive for hours or even days. This time lag, often referred to as communication latency or communication gap, can stall the entire bug-fixing process and cause a loss of momentum [42]. It also makes it difficult to have a nuanced, back-and-forth discussion, which is often necessary to fully understand and diagnose a complex issue. Korkala's study [80] shows that when the reliance on asynchronous tools increases, the software defect rates increase accordingly.

3 AI-Powered Bug Tracking Framework

To mitigate some of the problems of the traditional bug tracking systems detailed in Section 2, we propose a visionary bug tracking framework that leverages LLM/AI agents. We address some of the limitations highlighted in the previous section, including the subjectivity and inefficiency of manual bug reporting and classification, as well as the complexities of bug localization and the risk of regression during fix verification. By integrating LLM/AI agents into bug tracking, we aim to enable automation and intelligence at multiple stages of bug tracking. LLMs' role begins with assisting in bug report creation and enhancement and continues through reproduction, classification, traceability link creation, validation, bug assignment, localization, fixing, verification, and deployment.

3.1 Overview

As shown in Figure 2, the process begins with an interactive dialogue between the user and the chatbot, where the user submits a bug report and the chatbot engages with follow-up questions until sufficient detail is collected. After this point, the agents attempt to enhance the report to ensure clarity and completeness. Once the enhancement is completed, the agents repeatedly try to reproduce the bug. If reproduction is unsuccessful after a specified threshold of iterations, the task is escalated to customer support for manual reproduction.

Following reproduction, the agents classify the bug and then execute the bug-feature tracer to determine which feature the bug is related to. Once this tracing is complete, the agents evaluate the validity of the bug. If the bug is deemed invalid, the agents attempt to resolve it through a no-code fix. This resolution is overseen by

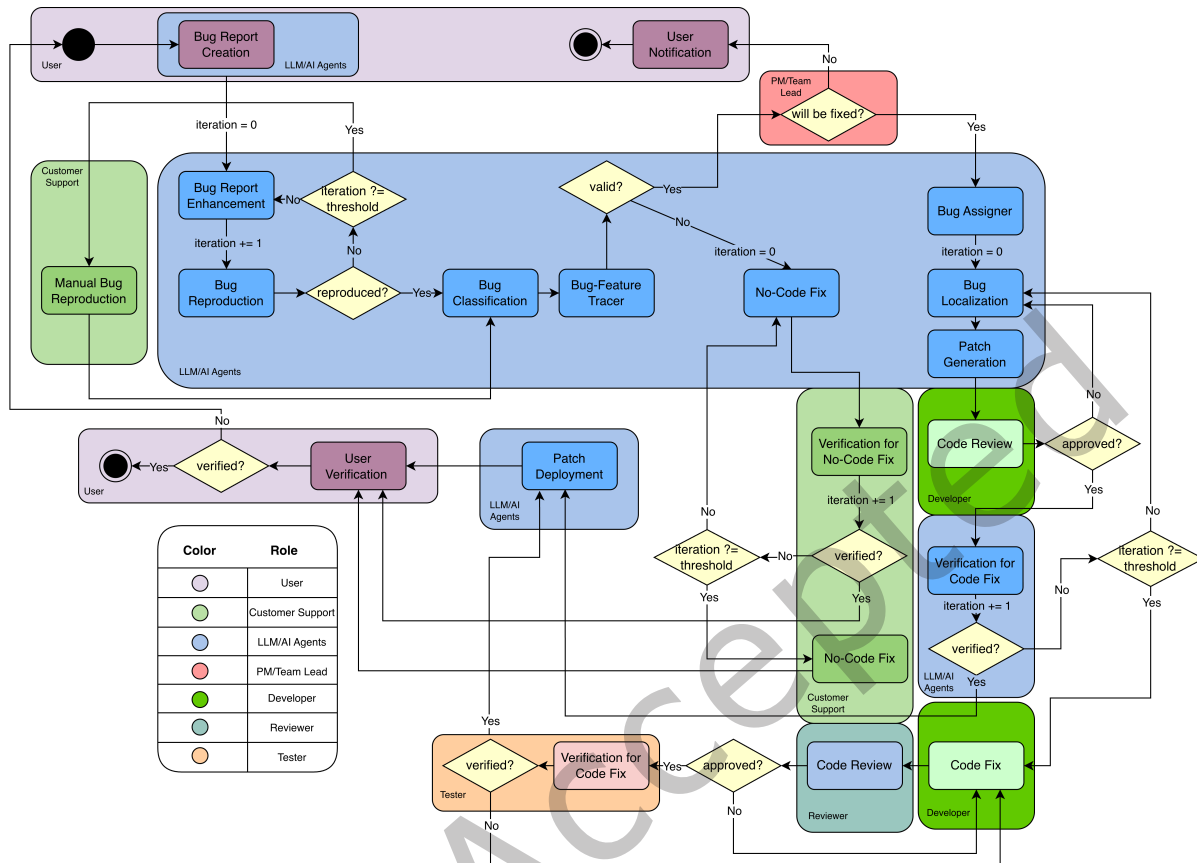


Fig. 2. Overview of Proposed Bug Tracking Framework Workflow

customer support to ensure accountability. If verification fails repeatedly and exceeds the predefined threshold, customer support provides a manual no-code fix. Once either the agent-based or manual fix is confirmed, the user is notified and asked to verify the resolution. If the user confirms the fix, the process ends; if not, the lifecycle restarts from the beginning.

If the bug is determined to be valid, the PM or team lead decides whether it should be fixed. If the decision is negative, the user is notified, and the lifecycle ends. If the bug should be fixed, the agents assign it to a developer, with the assignment reviewed by the team lead to ensure accountability. Even though agents may generate patches, human oversight ensures that a responsible developer is always associated with the bug to preserve accountability across the project.

Once assigned, the agents localize the bug and generate a candidate patch. The developer then reviews this patch. If the developer decides not to approve the generated patch, the agents repeat the localization and patch generation process until a valid solution is produced or a predefined threshold is reached. When the developer approves the patch, the verification phase is carried out by LLM/AI agents, which attempt to verify the fix within a specified threshold of iterations. If verification by the agent fails, the responsibility shifts back to the developer, who manually fixes the bug. This manual fix is then reviewed by another developer acting as a reviewer. If the

reviewer rejects the manual patch, the developer revises it until approval is achieved. The cycle resumes with the tester once the reviewer approves the patch. If the agents cannot again verify the bug within the threshold, the developer continues revising until successful verification is achieved.

When agent-based verification (or subsequent developer revisions) confirms the fix, the patch is deployed, and the user is once more asked for final verification. If the user accepts the fix, the lifecycle is completed. If the user rejects it, the process restarts from the beginning, ensuring continuous accountability, resolution, and user satisfaction. Finally, we note that duplicate bug detection is intentionally not modeled as a standalone stage in this lifecycle. Rather than representing a single linear step, duplicate detection functions as a cross-cutting capability that can be applied at multiple points in the workflow, including during initial report creation, reproduction, classification, and validation. Because its role is inherently supportive of several phases rather than constituting a distinct phase of its own, we integrate it implicitly throughout the pipeline instead of depicting it as a separate stage. The details of each phase in this lifecycle are elaborated in the following subsections, where we describe the design, methodology, and role of agents at every stage of the process.

3.1.1 Bug Report Creation. In modern software projects, bug reports remain the primary channel through which users communicate problems to developers. These reports are usually submitted through issue tracking systems such as Bugzilla, Jira, or GitHub Issues, where users manually fill out forms that include fields like a summary, description, S2R, and sometimes severity or priority levels. Empirical studies of large ecosystems such as Apache, Eclipse, and Mozilla show that this structured but largely manual approach to bug reporting has become standard practice in the industry [181]. More recent surveys confirm that bug reports today generally consist of textual descriptions, optional attachments such as screenshots, and structured fields that are later consumed by downstream tasks such as triaging or classification [100].

At the same time, the research community has begun to investigate new ways to facilitate bug report creation by reducing the burden on reporters and capturing more useful information from the outset. Several studies have highlighted that critical elements such as observed behavior (OB), expected behavior (EB), and S2R are often missing or only partially provided in bug reports, particularly for web applications [143]. To address these gaps, interactive systems have been proposed that actively guide users during the reporting process. For example, BugListener [121] demonstrated how informal conversations in collaborative live chats can be automatically transformed into structured bug reports, synthesizing OBs and EBs as well as S2R. Similarly, other research has shown that automatic follow-up questions can be selected when reports are incomplete, ensuring that reporters provide the information most useful for later reproduction [70].

Building on this line of work, chatbot-driven reporting has emerged as a promising paradigm. Systems like BURT [125] provide end-users with an interactive interface where a conversational agent guides them step by step, asking clarifying questions, suggesting graphical options, and providing instant feedback on whether their descriptions are clear and sufficient. A related study further demonstrated that such task-oriented dialogue systems can significantly improve the quality of bug reports compared to static template-based forms, especially when dealing with visually observable defects in mobile and GUI-based applications [126].

In our framework, we extend this trajectory by employing an LLM-powered chatbot as the entry point for bug report creation. Unlike static forms, a chatbot accepts natural language input from users and adaptively engages them with follow-up questions whenever essential details are missing. This direct interaction helps resolve the asynchronous nature of traditional bug reporting, where users often wait hours or days for feedback, by providing immediate responses and clarification in real time. For example, if a user reports, “the login page freezes after I click submit,” the chatbot may follow up by asking, “What did you expect to happen after clicking submit?” and receive the response, “I expected to be redirected to my account dashboard.” It can then request further context, such as the browser used or whether any error message appeared. At the same time, certain environmental details such as OS, app version, or device type can often be inferred automatically from metadata without needing to ask

the user directly. By the end of this short exchange, the chatbot compiles a structured report that contains the OB (page freeze), EB (dashboard redirection), and key environmental details (browser, OS, error messages), ensuring that the report is both complete and minimally burdensome for the user. This ensures that the key components of a bug report are systematically captured during the initial interaction. By grounding bug report creation in a conversational process, we aim to both reduce the cognitive effort for end-users and increase the completeness and usefulness of the reports passed downstream to subsequent phases of our framework.

3.1.2 Bug Report Enhancement. In our proposed methodology, end users create bug reports from scratch, as detailed in Section 3.1.1, with the support of an LLM assistant. Despite this guidance, bug reports may still contain ambiguities or missing information because end users are often non-expert reporters [125]. Therefore, after creation, bug reports are evaluated for completeness, clarity, and conciseness. Prior research has identified structural completeness, clarity, and the presence of core components as key dimensions for report quality [19]. Subsequent studies proposed automated techniques for detecting missing information, including incomplete S2R steps [25, 26, 124].

Recent studies have leveraged LLM capabilities to enhance bug report quality. LLM-driven approaches can semantically analyze bug reports and detect missing fields [104]. They can suggest enhancements for incomplete descriptions [23] or ambiguous content [45]. Critical information such as system state, error messages, and execution logs can be added by the agent [145]. The agent can also ensure S2R steps are complete [3]. Furthermore, LLMs maintain consistency with prior examples and best practices in bug reporting [123]. This process ensures that enhanced reports are actionable for developers [125].

In our methodology, the bug report enhancement process adopts this LLM-driven approach. The agent first analyzes the submitted report and evaluates its quality against criteria such as completeness, clarity, and conciseness that are defined by Bettenburg et al. [19]. Based on this assessment, the agent proposes enhancements, filling missing details or rephrasing ambiguous descriptions [3, 104]. Both the original and enhanced versions of the bug report are stored in the database, allowing developers or stakeholders to reference the original submission if necessary. For all downstream tasks in our proposed bug tracking workflow, such as bug reproduction, localization, and no-code fixes or patch generation, the enhanced version is used as input, as well as the original report, for developers' later references.

3.1.3 Bug Reproduction. In early systems, bug reproduction was manual: test engineers, customer support, or developers attempted to follow the S2R included in the report, often needing to contact users for clarification when the description was incomplete or ambiguous. Research gradually moved toward automation. Techniques emerged to extract reproduction steps from natural language descriptions [177] or to leverage runtime information such as logs, execution traces, or telemetry [142]. In mobile and GUI domains, specialized approaches replayed user reviews, event sequences, or monitored app interactions to recreate failures [95]. These works highlight how the community attempted to reduce reliance on manual effort, though success was often limited to narrow domains or specific application types.

With the rise of LLMs, bug reproduction has increasingly been reframed as a generative task. LLMs can read bug descriptions and produce candidate test cases or executable scenarios that explicitly trigger the reported failure, thereby supporting the reproduction of the underlying bug. LIBRO [77] demonstrated that LLMs are capable of generating test cases from natural language bug reports. Other studies integrated visual context: ReCDroid+ [179] and vision-based reproduction methods showed how screenshots and GUI interactions can be leveraged to systematically replay UI-intensive failures [144]. More recently, feedback-driven methods like ReBL [145] explored iterative prompting and adaptation, where an LLM interacts with execution feedback to refine its reproduction strategy over multiple rounds. BugCraft [167] extended this idea into the game domain, showing how iterative synthesis and refinement can be applied to highly interactive environments. Most recently, work at Google has shown that agentic BRT (Bug Reproduction Test) generation, where an LLM agent produces

fail-to-pass tests that both reproduce and validate bugs, can significantly improve industrial-scale automated program repair by providing stronger debugging and validation signals [32].

Building on this trajectory, our methodology operationalizes bug reproduction as an iterative loop tightly coupled with bug report enhancement. Once a bug report is enhanced, the agents attempt automated reproduction in a controlled sandboxed or containerized environment, ensuring determinism and isolation from external noise such as interrupts, concurrent kernel activity, and asynchronous hardware effects [28]. For GUI bugs, this involves simulating user interactions based on screenshots or videos [98, 178], while backend and API bugs may be reproduced by replaying logs or workloads reconstructed from user sessions [142]. If reproduction succeeds, the system generates an executable artifact that can be passed to verification and integrated into regression pipelines.

If reproduction fails, the system increments an iteration counter and loops back to the bug report enhancement phase. In this phase, LLM agents refine the S2R by reordering actions, adjusting parameters, adding missing environmental constraints, and, following established practices in prior work, actively requesting additional artifacts such as logs or screenshots [43, 45, 76]. The refined report is then re-executed, and this loop continues until reproduction succeeds or a threshold is reached. At that point, the case escalates to customer support for manual handling.

This iterative loop transforms reproduction from a one-shot attempt into a self-correcting process, where each failure strengthens the next attempt. Bug reports thus become dynamic artifacts that evolve through agentic refinement until a reproducible scenario is achieved. By feeding these successful reproductions into regression test suites, our framework creates an expanding safety net to prevent regressions in future builds [104].

3.1.4 Bug Classification. Once a bug is successfully reproduced, the next step is classification. Traditional bug classification typically requires substantial manual intervention to evaluate factors such as severity, impact, and priority [9]. Practitioners look for cues in the bug report, such as affected features, system logs, and error messages, to gauge how urgently a fix is needed. This manual process can be time-consuming and prone to human error, as it often depends on individual judgment and expertise.

Beyond severity and priority, the classification may follow various taxonomies based on the bug’s nature—e.g., functional, performance, UI, or security [41]. These categorizations help teams allocate appropriate resources (e.g., security experts for security bugs or UX designers for UI issues) and streamline the handoff process between different roles within an organization.

Recent research has explored the use of ML [83, 85] and LLMs [41, 81, 93] to automate bug classification by analyzing bug report text to infer severity, category, and potential root causes. Mashhadi et al. [105] show that using fine-tuned CodeBERT for bug severity prediction improves results by 29%-140% for several evaluation metrics, compared to classic ML prediction models. In addition, an LLM-based bug-fixing time prediction classifier [11] could be helpful while assigning priority and severity.

In our proposed framework, we predict the priority, severity, and type of the bug before checking the trace link using previously studied ML/LLM classifiers. Such AI-driven approaches can systematically process bug reports to provide near real-time classification, thereby decreasing TTR.

3.1.5 Bug-Feature Traceability. In the literature, several traceability link categories are available using different artifacts such as requirements [5, 56], software documentations [6, 46], test cases [47], and issues [103, 168]. ML models [47] and LLMs [5, 60] are being used to achieve different traceability tasks.

Building on these advances, our framework proposes to maintain bug-feature trace links, explicitly connecting each reported defect to the product feature it affects. The primary benefit of this form of traceability is the ability to contextualize and prioritize defects within the broader product architecture. By automatically mapping a bug to the specific feature it impacts, development teams gain a clearer understanding of the bug’s scope and potential consequences. This connection supports more informed decision-making: project managers can

accurately identify which features are most problematic, allocate resources where they are most needed, and ensure that critical issues affecting key functionality are addressed promptly.

In addition, bug–feature traceability creates a valuable historical record. Over time, this repository of links enables trend analysis, revealing features that are consistently error-prone and guiding long-term architectural improvements or targeted testing strategies. Such trace links also strengthen downstream activities such as release planning, regression testing, and impact analysis, since developers can quickly determine which features are likely to be affected by a change or a newly discovered defect.

Automating this process minimizes the risk of human error and overlooked relationships that commonly occur in manual triage. It also facilitates cross-team communication by providing a shared, up-to-date view of the relationship between bugs and features. Ultimately, maintaining accurate bug-feature trace links supports a more streamlined and data-driven defect resolution process, helping to reduce maintenance costs and improve overall product quality.

3.1.6 Bug Validity Check. After a bug has been successfully reproduced and classified, the agents evaluate its validity before proceeding further in the lifecycle. Not all reported issues correspond to genuine software defects; some may stem from misunderstandings of EB, incorrect configurations, or environment-specific constraints [132, 155]. The bug validity check ensures that only issues requiring a code-level fix are advanced for resolution, while invalid reports are categorized appropriately, thereby avoiding wasted development effort.

Early studies have demonstrated that analyzing S2R can effectively distinguish between valid and invalid bug reports [44]. Other works extended this by leveraging system logs and error messages to identify whether reports contained sufficient technical detail for validation [62, 88]. More recently, research has examined how noisy and incomplete logs in industrial contexts complicate this process and developed methods to handle such cases [87]. In parallel, approaches that exploit semantic and contextual information from bug descriptions have shown promise in improving classification and filtering accuracy [106].

Building on these studies, our proposed workflow employs an LLM agent to perform the validity check by analyzing S2R, logs, and error messages while also handling noise and ambiguity in real-world reports. The LLM agent can also use traditional ML models to make a decision about the bug report’s validity. To further strengthen its decision-making, the agent cross-references historical bug reports and project documentation in a retrieval-augmented generation (RAG) setting as in Dinç and Tüzün’s study [39].

If the reported behavior corresponds to an intended feature or an expected system outcome, the agent classifies the bug report as invalid. Invalid bugs are then mapped to a structured taxonomy like user error, duplicate report, configuration error [139], and the workflow continues with the no-code fixes targeting such cases. As part of this process, the LLM agent also generates a natural-language explanation for why a report was labeled as invalid. This explanation can later be consumed by other agents in the workflow, such as those generating no-code fixes. Also, this explanation can be used and overwritten by the customer support staff while deciding about validity, as the customer support staff supervises this process. Finally, this explanation can also be provided to end users, thereby improving transparency and user trust.

If the bug is deemed valid, responsibility shifts to the PM, who decides whether the bug will be fixed. If the PM determines that the bug should not be fixed, the report is labeled as *Won’t Fix*, and the user is notified of the rationale behind this decision.

By incorporating an agentic bug validity check, our proposed bug tracking framework will effectively filter out false positives and ensure that only actionable bugs are advanced to PM and developers. This agentic validation mechanism would not only improve the precision of bug tracking but also contribute to more efficient use of engineering resources, minimize human labor, solve class imbalance problems that frequently occur in ML algorithms, and decrease TTR.

3.1.7 Bug Assigner. After the bug’s validity is checked, the next step is assignment. In our methodology, if the bug is not valid, it is still routed to the no-code fix stage handled by LLM agents, which is explained in the next subsection. At the same time, the bug is assigned to a customer support representative to ensure oversight and accountability, so that at least one person remains responsible for the bug until it is formally closed. In practice, there could be multiple customer support representatives available, but for simplicity, our current assumption is that there is only one, and the invalid bugs are consistently assigned to that individual. If the bug is valid, the PM or Team Lead first determines whether it should be fixed. For bugs approved for fixing, the bug assigner agents automatically recommend the most suitable developer, and these recommendations are then reviewed by the PM or Team Lead before the bug moves to the fixing stage.

The task of bug assignment has historically been studied under the umbrella of automated bug triage. Early research treated assignment as a text classification problem, using techniques such as TF-IDF [127] representations and Naive Bayes classifiers to route bugs to developers [34]. Semi-supervised approaches extended this line of work by leveraging unlabeled bug reports to improve classifier performance [164]. Large-scale empirical studies examined how these models perform in industrial contexts, showing both potential and practical challenges [37]. More advanced approaches modeled developer behavior through bug tossing graphs to reduce reassignment and misallocation [21]. Topic models were also employed to capture latent semantics in bug reports and better align them with developer expertise [159, 166]. Other frameworks explicitly considered developer interest and workload in the assignment process [17]. While these approaches significantly reduced the manual effort involved in bug triage, their reliance on shallow features and frequent retraining limited their scalability and adaptability.

With the advent of deep learning and LLMs, a new generation of approaches emerged. Transformer-based methods such as BERT [38] have been applied to bug triage, enabling richer semantic representations of bug reports and improving accuracy [92]. Ensemble approaches that combine multiple language models have further improved robustness and prediction performance [86]. Beyond software maintenance, similar ideas have been explored in incident management: COMET demonstrated that LLMs can provide accurate and interpretable assignment of cloud incidents at Microsoft, showing that LLMs can reliably map natural language reports to the right resolver [152].

Our methodology builds on these advancements by introducing AI-powered bug assigner agents that act as an integral part of the bug tracking system. Once the decision to fix has been made, the agents automatically identify the most appropriate developer based on semantic similarity to historical bug assignments, workload balancing, and alignment with project policies. Assignments are generated quickly and consistently, minimizing costly bug tossing and delays. Importantly, the PM or Team Lead remains in the loop as a reviewer, ensuring accountability and organizational oversight.

3.1.8 Bug Handling with No-Code Fixes. If a bug report is deemed invalid, it often requires no-code fixes. No-code fixes resolve such issues by making changes outside of the application’s source code, typically involving adjustments to configuration files, server settings, user permissions, or system-level parameters. In traditional bug tracking platforms, these fixes are usually proposed and applied by customer support staff rather than PMs or developers, since they often relate to usage errors, misconfigurations, or environment-specific issues rather than programming defects.

The proposed approach builds on a growing body of research in automated configuration troubleshooting and repair. Early works such as PeerPressure [146] leveraged statistical comparisons across systems to detect anomalous configuration states, while AutoBash [129] introduced speculative execution of configuration changes with rollback guarantees. More recent approaches, including ConfAid [14] and range-based fix generation [160], have shown that dynamic analysis and constraint-based reasoning can effectively localize and repair misconfigurations. Recent studies also explore the use of LLMs for automated repair of container and cloud configuration errors [169], highlighting the increasing feasibility of applying LLM-driven agents to this domain.

In our proposed workflow, no-code fixes are automatically recommended by the LLM agent. The agent analyzes the S2R, logs, and error messages provided in the bug report, and cross-references them with historical bug data and project documentation in a RAG setting. For invalid bugs, the agent proposes minimal, targeted adjustments such as correcting environment variables, updating database connection strings, revising access permissions, or tuning server timeout values. In this HIL setting, customer support staff act as supervisors for this step, reviewing the no-code fixes and modifying necessary parts.

To ensure that proposed fixes do not inadvertently introduce new defects, our bug tracking system employs a generate-and-validate process in an HIL setting. First, the LLM agent localizes the relevant configuration options associated with the reported failure. Next, it generates one or more candidate adjustments, informed by both learned patterns and historical resolution strategies. The effectiveness of these adjustments is assessed using automated regression tests, integration checks, or environment-specific predicates. If a candidate fix resolves the bug without introducing side effects, it is directed to the end user under the supervision of the customer support staff.

3.1.9 Bug Localization. Once a PM approves a bug for fixing and confirms the assignment, the process of bug localization begins. Historically, this has been a challenging task, as developers must manually sift through vast and complex codebases to find the exact line of code or code chunk responsible for an issue [49]. Traditional bug localization techniques, such as Information Retrieval-Based Fault Localization [158] and Spectrum-Based Fault Localization [153], have been effective, but they are often complemented by other methods. Slicing-based fault localization [154] narrows down the search space by identifying relevant parts of the code that could influence the program’s behavior, while Mutation-based fault localization [48] generates a set of mutants (tiny, seeded bugs) to see if they can be killed by the failing test cases, thus helping to pinpoint the real bug. Still, these methods often rely on statistical analysis of test coverage rather than a deep understanding of the code’s semantic meaning.

The recent integration of LLMs has introduced a new paradigm, significantly enhancing debugging efficiency by going beyond traditional methods. Researchers have proposed various frameworks and systems to leverage LLMs for localization [40, 74, 96, 165]. For instance, the FlexFL framework [162] introduces a two-stage approach. The first stage uses traditional fault localization techniques to narrow down potential bug locations, and the second stage uses an LLM-based agent to perform a deeper analysis of the reduced search space. Similarly, SOAPFL [117] formalizes the bug localization process into a procedure that mimics a human developer’s workflow, comprising comprehension, navigation, and confirmation steps. These studies showcase how LLMs can directly understand the semantic meaning of the code itself, enabling a more intuitive and accurate approach to finding the root cause.

Our system will use LLMs to analyze not only source code but also execution traces, system logs, and stack traces to determine where a failure originates. By leveraging historical debugging data and dependency graphs, the system will pinpoint the most likely areas of failure and reduce the search space for developers. This automation ensures that once a bug is approved for fixing, developers receive clear, context-aware insights into its root cause, significantly improving efficiency in the localization process.

3.1.10 Patch Generation. Once a bug has been localized, developers receive AI-assisted recommendations for potential fixes. While a bug’s root cause may be understood, the actual process of changing the code is not trivial, as it can introduce new issues or ripple effects in other parts of the system. Historically, automated program repair (APR) has sought to address this by using techniques such as generate-and-validate approaches. Early methods like GenProg [91] showed promise but were often limited by the randomness of their mutation operations, which could lead to nonsensical or unmaintainable patches. Other approaches, such as Prophet [99], improved upon this by learning a probabilistic model from a dataset of human-written patches to rank potential fixes. Despite these advances, these methods often struggled to generalize and faced challenges in understanding the broader semantic context of a bug.

The recent advancements in AI-driven patch generation have revolutionized this part of the bug-fixing process, and we would utilize existing studies to establish our semi-automated bug tracking system. This is a well-studied field, with a growing body of research demonstrating how LLMs can automatically generate code fixes for a wide range of bugs [67, 75, 148, 157].

In our proposed framework, the LLM generates multiple candidate patches, each of which is reviewed by a developer or designated reviewer. If a patch is approved during this review, it is merged; otherwise, the generation–verification cycle is repeated. If the number of unsuccessful cycles goes beyond the set threshold, the developer manually creates a fix.

By using this approach, instead of manually editing the code, developers can review and refine AI-generated suggestions, freeing them to focus on more complex, creative tasks. By minimizing human error and proactively suggesting optimal fixes, the system ensures that the final product is more robust and secure.

3.1.11 Bug Fix Verification. In the bug fix verification step, our system assesses the correctness of generated patches and iteratively refines them to ensure reliability and semantic correctness. Foundational work in APR highlighted the importance of rigorous verification to move candidate patches from plausibility to practical adoption [116, 161].

Recent studies have explored leveraging LLMs for patch generation and validation, as well as verification. LLM-driven approaches can generate test cases to verify candidate patches [94], detect semantic deviations [149], and iteratively refine patches based on verification feedback [90]. These methods demonstrate the potential of LLMs to improve patch correctness, coverage, and production readiness.

Building on these insights, our methodology integrates an LLM agent into the bug fix verification workflow. Crucially, this agent is designed not merely to act as a passive test runner, but as a rigorous evaluator that actively examines patches for deceptive or trivial fixes. To prevent the generation of trivial or deceptive fixes, such as simply removing a failing assertion or skipping a test, the system enforces strict constraints, such as existing tests cannot be deleted or modified without explicit human authorization.

Furthermore, the agent employs active test augmentation to ensure the patch is not merely overfit to the specific failure. By generating new test cases that target the modified code paths, the system verifies that the patch preserves existing functionality and does not introduce regressions. In this HIL setting, test engineers supervise the verification process, reviewing the results and ensuring that candidate patches meet quality standards. Until a predefined number of iterations threshold is reached, our proposed workflow reverts to bug localization and generates new patches in cases of verification or regression testing failures. This approach bridges the gap between APR and practical patch adoption, moving beyond syntactic plausibility toward semantically correct, production-ready fixes. If the agent cannot verify within a specified number of iterations, the verification task would be delegated to the test engineer.

3.1.12 Patch Deployment. Patch deployment in contemporary software organizations is typically realized through CI/CD pipelines, supported by toolchains that automate the path from code to production. In practice, once a patch is verified and merged, CI/CD platforms such as Jenkins²⁵, GitHub Actions²⁶, or GitLab²⁷ CI trigger build processes, package the code, run integration and regression tests, and then promote the artifacts to staging or pre-production environments. From there, deployment into production may follow patterns such as blue–green deployments or canary releases, which reduce risk by gradually exposing the patch to subsets of users before a full rollout. Industry studies highlight that these practices significantly accelerate release frequency and improve reliability. For instance, case evidence from Paddy Power’s adoption of continuous delivery demonstrates that

²⁵<https://www.jenkins.io/>

²⁶<https://github.com/features/actions>

²⁷<https://about.gitlab.com/>

releases shifted from a few times per year to weekly or even daily, with deployment becoming a single-button operation instead of a multi-day manual task, leading to dramatic improvements in productivity, quality, and customer satisfaction [30]. Likewise, a multiple-case study of Finnish software-intensive companies shows that organizations with more automated toolchains, especially those minimizing manual steps, are able to deploy faster and more reliably, while gaps in automation, such as missing performance testing or acceptance testing, directly slow down the delivery cycle [109].

Despite this progress, patch deployment still remains largely a DevOps responsibility, with human operators accountable for execution and rollback in case of failure. LLMs have not yet been used to autonomously perform deployment, but they open new opportunities to augment the process. LLMs can generate deployment scripts, infrastructure-as-code specifications, and container orchestration files, reducing the manual effort required to configure environments. They can also enhance monitoring by analyzing telemetry and log data post-deployment, quickly surfacing anomalies or regressions that would otherwise require manual inspection. Furthermore, LLMs could assist in adaptive rollout strategies by reasoning over user feedback and operational metrics to recommend accelerating, pausing, or rolling back deployments. However, accountability cannot be transferred to an automated agent, so deployment remains a hybrid process where LLMs act as copilots rather than autonomous actors.

Our methodology builds directly on these insights. When the tester or LLM/AI agent verifies the bug, the deployment itself continues to be executed by the CI/CD infrastructure and operations teams, but the LLM acts as an intelligent assistant across the deployment lifecycle. It prepares deployment descriptors alongside generated patches, reasons about potential risks before rollout, and provides continuous monitoring support after deployment. In doing so, it reduces manual effort, accelerates the transition from patch generation to user verification, and integrates deployment more tightly into an LLM-augmented bug tracking lifecycle. Following successful deployment, the system immediately involves the end user in the final verification step. The user receives a summary of the deployed fix and is prompted to confirm whether the issue has been resolved satisfactorily. If the user accepts, the bug is closed and marked as resolved; if not, the process loops back to the reporting stage with enriched details, ensuring continuous accountability and refinement until the user is satisfied.

3.2 Architecture of the Proposed Bug Tracking System

Our proposed bug tracking system employs a multi-LLM agent design within a structured layered architecture. In this design, specialized LLM agents are assigned to discrete steps in the bug tracking lifecycle. Rather than relying on a single monolithic model to handle all tasks, each agent is optimized and prompted for a particular subtask, such as historical bug fixes for localization or no-code fix suggestions. This decomposition enables each agent to use toolchains best suited to its objective, retain a compact, task-relevant context for efficiency, and produce more focused, explainable outputs. To ensure seamless interoperability between these specialized components, agents communicate via a standardized Agent-to-Agent (A2A) protocol [133], which defines the contract for data exchange and state handoffs.

To manage the complexity of multiple data sources, toolkits, and agents, we adopt the Model Context Protocol (MCP) [82] pattern. This approach centralizes context management and tool integration, allowing the agent microservices to remain lightweight and interchangeable while ensuring consistent access to external resources and LLM APIs.

Figure 3 visualizes this layered architecture, organized into five distinct horizontal levels:

- (1) **External Systems Layer:** Located at the top, this layer handles interactions with the outside world, including the *Bug Reporting Client* for user inputs and the *LLM API Provider* that powers the intelligence of the system.
- (2) **Platform Orchestration Layer:** This layer serves as the system’s central nervous system. It contains the *Workflow Orchestrator*, which manages the sequence of agent execution and state transitions adhering to

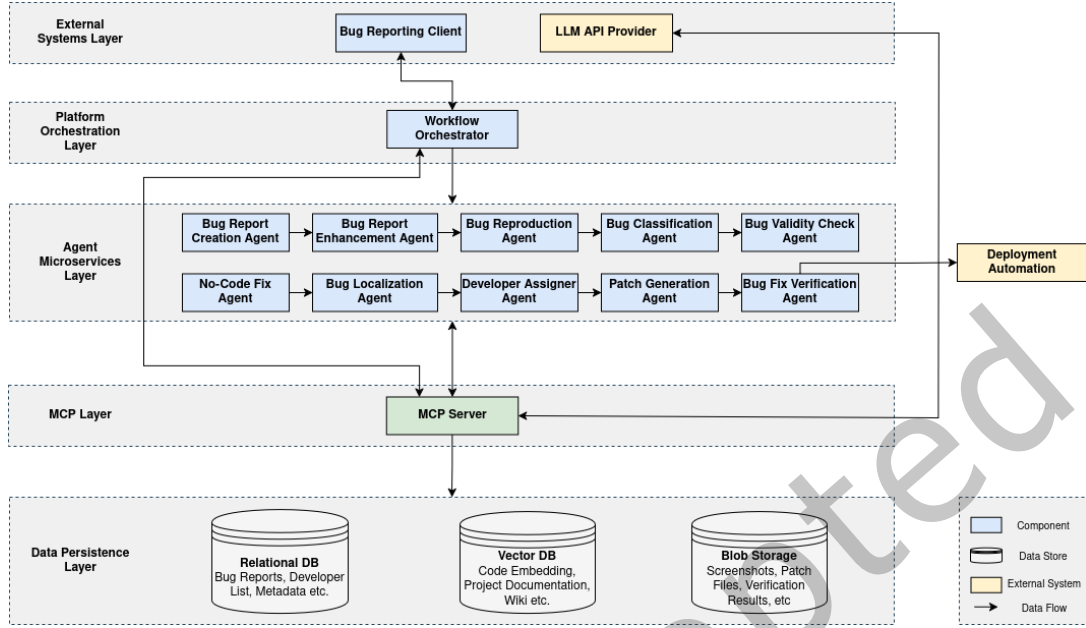


Fig. 3. Architecture of Proposed Bug Tracking Framework

the A2A protocol. It orchestrates the flow of control and data between the upper client layers and the specific agent microservices.

- (3) **Agent Microservices Layer:** This layer hosts the sequence of specialized agents operating under the A2A protocol. The workflow begins with the *Bug Report Creation* and *Enhancement Agents*, moving through *Reproduction*, *Classification*, and *Validity Check Agents*. Subsequently, the pipeline triggers the resolution phase with the *No-Code Fix*, *Bug Localization*, *Developer Assigner*, *Patch Generation*, and *Bug Fix Verification Agents*. Successful verification leads to the external *Deployment Automation* stage.
- (4) **MCP Layer:** Situated centrally between the agents and the data stores, this layer functions as the integration broker. The *MCP Server* mediates access between the agents in the layer above and the external systems or data layers below. It standardizes prompt templates and tool integrations, while managing the connection to the *LLM API Provider* to supply the agents with inference capabilities.
- (5) **Data Persistence Layer:** At the bottom, the architecture utilizes specialized storage solutions to maintain system state and knowledge. A *Relational DB* stores structured data such as bug reports, developer lists, and metadata. A *Vector DB* handles semantic data, including code embeddings, project documentation, and wikis. Finally, *Blob Storage* is used for unstructured artifacts like screenshots, patch files, and verification results.

In summary, the architecture provides a structured way to operationalize multiple LLM agents. By organizing the system into logical layers—separating orchestration, agent logic, context management, and data persistence—we gain modularity, scalability, and governance. The A2A protocol, combined with the distinct MCP layer, ensures that, despite the distributed nature of the microservices, data access, inter-agent communication, and tool usage remain consistent and auditable. Remaining system-level challenges include ensuring state consistency across the orchestration layer and managing the latency of sequential agent interactions. Addressing these will be key in future work to make multi-LLM and multi-agent-powered bug tracking systems robust and production-ready.

3.3 Early Adaptations of Proposed Framework

To provide feasibility-oriented empirical evidence for our proposed framework, we report results from a few implemented subcomponents, including modules introduced and evaluated in our own prior work. We first summarize empirical findings on early stages that are central to real-world bug tracking, bug report enhancement, reproduction from natural-language reports, and validity detection. We then situate these findings alongside representative recent systems that empirically demonstrate the feasibility of downstream repair stages such as localization, patch generation, and verification.

In the domain of bug report enhancement, "ImproBR" is introduced, which is an agentic framework designed to transform ambiguous, user-generated bug reports into structured, actionable documents by refining critical sections such as S2R, OB, and EB [3]. The methodology employs a multi-stage pipeline that first utilizes a fine-tuned DistilBERT classifier and heuristic analysis to detect missing or low-quality sections, followed by a "Bug Report Improver Agent" powered by GPT-4o mini. To ensure domain accuracy and minimize hallucinations, the agent leverages RAG to access knowledge from the Minecraft Wiki. The evaluation of ImproBR was conducted on the Mojira²⁸ dataset. Results demonstrate substantial improvements in both structure and content quality. First, the system increased the average structural completeness of reports from 7.9% to 96.4% by automatically generating missing S2R, OB, and EB sections. In terms of content richness, the average word count for S2R sections rose from 9.1 to 111.9, and the usage of domain-specific terminology increased from 0.36 to 4.59 per report. Furthermore, the semantic similarity of generated S2R sections to ground-truth duplicate reports improved from 0.020 to 0.180, indicating a closer alignment with expert-written content. Crucially, a manual evaluation of 139 challenging, real-world bug reports confirmed the practical impact of these enhancements. ImproBR more than doubled the proportion of executable S2R sections, increasing the average from 28.8% to 67.6%, and successfully raised the number of fully reproducible bug reports from 1 to 13.

Focusing on bug reproduction, "BugCraft" is introduced, an end-to-end automated framework capable of reproducing crash bugs in the complex, open-ended environment of Minecraft directly from user descriptions [167]. The system's methodology comprises two distinct stages: a "Step Synthesizer" that uses LLMs augmented with RAG to convert unstructured reports into high-quality, clustered S2R, and an "Action Model" vision-based agent leveraging a custom macro API and OmniParser to execute these steps within the game client. The system is evaluated on "BugCraft-Bench," a curated dataset of 86 reproducible crash reports, where BugCraft achieved a state-of-the-art success rate of 34.9% using GPT-4.1, significantly outperforming general computer-use baselines like OpenAI's CUA (25.5%) and UI-TARS (0%). Additionally, the system proved highly efficient, reducing the cost per attempt to approximately \$1.16 compared to the significantly higher costs of manual reproduction, as well as a decrease in TTR from manual 3.41 days to automated 10 minutes.

Regarding the bug report validity detection, we conducted a study to assess the efficacy of various models in classifying bug reports as valid or invalid [39]. The methodology involved training and evaluating a spectrum of models including classical ML algorithms (SVM, Random Forest), fine-tuned transformers (RoBERTa, BERT), and reasoning LLMs (GPT-o3-mini, Llama-3) using a dataset of 10,000 Firefox bug reports; we also introduced a novel "Judge LLM" ensemble that combines the probabilistic votes of classical classifiers with RAG-retrieved few-shot examples to render a final decision and explanation. Our results established that fine-tuned semantic classifiers are superior, with RoBERTa achieving the highest F_1 score of 0.909, whereas standalone LLMs lagged behind but showed dramatic improvement when augmented with RAG (rising to an F_1 of 0.815). Uniquely, the Judge LLM pipeline achieved a competitive F_1 of 0.871 while providing valuable natural language explanations for its verdicts.

In the domain of automated bug-fix verification, "FIXPAD++" is introduced as a novel framework that verifies patches for GUI-based desktop applications exclusively through visual interaction, without requiring access to

²⁸<https://bugs.mojang.com/browse/MC>

source code or internal instrumentation [71]. The methodology employs a two-phase pipeline: a Reproduction Phase and a Verification Phase. During reproduction, a multi-modal multi-agent system—comprising specialized Action, Observation, and Reflection agents—interacts with the buggy software version using an extended ReAct paradigm augmented with explicit self-reflection. This system utilizes OmniParser V2 to transform screenshots into a structured GUI state and is powered by Gemini 2.5 Flash to generate a crash-inducing action trajectory. Upon successful reproduction, the Verification Phase utilizes a trajectory replay mechanism to execute the recorded action sequence on the patched version to assess whether the issue has been resolved. FIXPAD++ was evaluated on "FIXPAD-BENCH", a dataset consisting of 105 evaluation instances derived from 22 real-world Notepad++ crash bugs, featuring both valid and invalid patches. The evaluation results demonstrate a reproduction success rate of 72.73% with an average reproduction time of 174.07 seconds. Among the successfully reproduced cases, FIXPAD++ correctly verified valid fixes with 87.50% accuracy and detected invalid fixes with 77.05% accuracy. In a comparative analysis against OpenAI's CUA, FIXPAD++ significantly outperformed the baseline, which achieved a 50.00% reproduction rate and a 40.91% correct fix verification accuracy. Furthermore, the replay-based verification phase proved highly efficient, completing in an average of 17.59 seconds per version, compared to the 174.07 seconds required for the initial agent-driven reproduction. By reusing identified crash trajectories as fixed test cases, FIXPAD++ provides a source-code-independent solution that successfully bridges the gap between GUI-level crash reproduction and automated patch correctness assessment for closed-source applications.

In addition to our own early implementations, several recent LLM-based automated software engineering frameworks provide strong empirical evidence that the core components of our proposed framework are already technically feasible at scale. While some of these systems adopt fully agentic control loops and others deliberately avoid persistent agent state, together they demonstrate that complex multi-stage bug resolution pipelines can already be constructed with current LLM-based technologies.

Agentless [156] presents a repository-level automated bug-fixing system that intentionally avoids maintaining persistent environment state or long-horizon agent memory. Instead, it relies on structured reasoning, static repository analysis, and targeted external tool calls. The system decomposes bug fixing into four major stages: issue understanding, file-level localization, patch generation, and verification. For localization, Agentless applies hierarchical reasoning over repository structure and file contents; for patch generation, it uses LLM-guided program transformation; and for validation, it executes candidate patches against available test suites. On the SWE-bench benchmark, Agentless achieves a verified fix rate of 27.8%, substantially outperforming many prior systems that rely on heavier infrastructure. Notably, the system maintains high efficiency, solving tasks with significantly fewer tool calls and reduced runtime overhead. These results demonstrate that even without a fully agentic architecture, sophisticated multi-stage bug resolution pipelines are already realizable, directly supporting the feasibility of our framework's design.

AutoCodeRover [175] proposes a fully automated program repair agent built around an iterative generate–execute–repair loop. The system first performs hierarchical localization, identifying suspicious files and functions using repository analysis and LLM-guided reasoning. It then synthesizes candidate patches and validates them through execution-based testing, refining unsuccessful patches across multiple iterations. On the SWE-bench-Verified benchmark, AutoCodeRover achieves a state-of-the-art verified fix rate of 46.2%. In ablation studies, removing execution-based validation drops performance by over 15 percentage points, highlighting the importance of tight integration between generation and verification. AutoCodeRover also demonstrates strong efficiency: for many tasks, it converges to correct patches within 3–6 iterations, showing that agentic feedback loops can quickly stabilize toward valid solutions. These findings closely mirror the patch generation and verification stages of our proposed framework.

OpenHands [150] presents a general-purpose software engineering agent designed to execute complex multi-step development workflows. Its architecture integrates planning, tool usage, repository navigation, and execution

feedback into a unified decision-making loop. The CodeActAgent variant performs best, combining code generation with explicit action planning and execution. On USEbench, a benchmark consisting of 1,271 real-world repository-level tasks, CodeActAgent achieves an overall task success rate of 26.8%. On software maintenance subsets of the benchmark, including bug fixing and feature implementation, success rates exceed 30%. The system also demonstrates strong generalization across programming languages and project scales, providing concrete evidence that LLM-driven agents can coordinate complex end-to-end software workflows similar to those envisioned in our framework.

USEagent [10] proposes a unified agent architecture capable of handling a broad spectrum of software engineering tasks, including bug fixing, regression testing, code generation, and test generation. The agent integrates hierarchical planning, long-horizon reasoning, repository exploration, and execution feedback into a single control loop. On USEbench, USEagent achieves 33.3% overall task efficacy, significantly outperforming OpenHands (26.8%). On software maintenance tasks derived from SWE-bench-Verified, USEagent reaches 45.6% verified fix rate, approaching the specialized performance of AutoCodeRover (46.2%), while maintaining substantially broader task coverage. Importantly, USEagent exhibits strong stability: over 70% of successful tasks converge within five agent iterations, indicating that agentic workflows can remain both scalable and predictable in practice.

While the above systems demonstrate impressive capabilities in automated localization, patch generation, and verification, they all operate under the assumption that the bug is already well-formed, reproducible, and ready for repair. In contrast, our framework is explicitly designed to handle the practical challenges of real-world bug tracking, where bug reports are often incomplete [174], ambiguous [20], invalid [155], or non-reproducible [53]. Rather than assuming perfect inputs, our system actively improves bug reports through interactive enhancement, attempts automated reproduction from natural language descriptions, classifies bugs, performs bug-feature tracing, supports no-code fixes for invalid reports, assigns bugs with accountability constraints, and integrates patch deployment and user verification into a unified lifecycle. By covering the entire pipeline from raw user submission to final resolution and feedback, our framework addresses substantial portions of the bug lifecycle that existing systems intentionally leave outside their scope.

Taken together, these systems demonstrate that the major components of our proposed bug tracking framework, automated localization, patch generation, iterative verification, multi-stage reasoning, and workflow orchestration, are already realizable with current agentic technologies. When combined with our own early results on bug report enhancement, bug reproduction, and validity detection, these findings provide strong empirical grounding for the feasibility of an end-to-end AI-powered bug tracking framework.

In the commercial landscape, the shift from passive issue tracking to AI-native automated resolution is already underway. A new generation of AI-powered debugging and maintenance platforms provides industrial validation for different phases of our proposed framework. Here, we show some of the recent developments in the industrial landscape.

Regarding the creation and enhancement of bug reports, industry tools have largely deprecated the manual entry of environment details. Platforms such as BetterBugs²⁹, Jam³⁰, Bugpilot³¹, and LogRocket³² have operationalized the concept of a bug creation agent. These tools automatically capture comprehensive telemetry, including console logs, network requests, device specifications, and session replays, at the moment of failure, eliminating the need for back-and-forth communication between users and developers. For instance, Bugpilot combines user-side reporting with automated error monitoring to capture “hidden bugs” that users fail to report, while Jam integrates directly with issue trackers like Jira to inject this rich context into the developer’s existing workflow.

²⁹<https://www.betterbugs.io>

³⁰<https://jam.dev>

³¹<https://www.bugpilot.com/>

³²<https://logrocket.com/>

For bug reproduction and localization, the industry is moving toward deterministic replay and automated test generation. Replay.io³³ and rrweb³⁴ record execution traces (DOM updates, network events) to render bugs deterministically replayable. Similarly, QA Wolf³⁵ demonstrates the viability of transforming user flows into structured, maintainable end-to-end test suites, effectively bridging the gap between a bug report and a reproducible test case.

Finally, regarding bug fixing, several platforms are deploying agents that mirror the *Bug Fix* and *Verification* components of our framework. TraceRoot.AI³⁶ and Keystone³⁷ connect directly to observability data (logs and traces) to construct execution trees, identify root causes, and automatically generate draft PRs to resolve production issues. Jazzberry.ai³⁸ further validates the verification aspect by executing code changes in secure, sandboxed microVMs to detect runtime errors before they reach production.

While these commercial solutions demonstrate the industrial viability of individual agentic capabilities, they currently lack rigorous results to be presented here, and they exist as fragmented point solutions, each handling either reporting, reproduction, or fixing in isolation. Our proposed framework unifies these disparate capabilities into a single, cohesive multi-agent orchestration, managing the entire lifecycle from the initial user report to the verified deployment of the fix.

3.4 Roles and Responsibilities of Each Stakeholder

Since we propose a new workflow for bug tracking, which integrates LLMs, the traditional roles and responsibilities of stakeholders participating in bug tracking are subject to change. This is because, with this new HIL paradigm, the workload shifts from being heavily dependent on manual effort toward a more balanced model, where LLMs perform core tasks under human supervision and guidance. Table 2 provides a summary of the evolving responsibilities, followed by a detailed explanation of each role transformation.

End User: In traditional settings, end users manually report bugs through email, ticketing systems, or predefined templates. They later confirm whether the fix resolves the problem or report if parts of the solution remain incomplete. In the proposed workflow, end users interact with an LLM-powered chatbot interface to report bugs. The LLM agent guides them by asking clarifying questions and ensuring that all required information is collected at the time of submission. End users are also notified about fixes through their preferred communication channel.

Customer Support: In the traditional bug tracking process, customer support staff receive bug reports and classify them as valid or invalid. They manually attempt to reproduce the bug using the provided S2R. If a bug is invalid, they may propose a no-code fix. They also act as intermediaries, communicating with both developers and end users regarding validity, status, and missing details. In the proposed workflow, customer support primarily acts as supervisors within the HIL workflow. Automated modules validate bug reports, attempt reproduction using provided S2R, and even propose no-code fixes when applicable. LLM agents communicate with end users about the process and status, while human staff oversee these activities, intervening when the automation fails or produces ambiguous results. Customer support staff also acts as testers for no-code fixes suggested by LLM agents, by executing tests for no-code fixes.

Project Manager/Team Lead: In traditional workflows, PMs decide whether a bug should be fixed, assign it to developers, prioritize its urgency, set deadlines, and track progress. They oversee the overall bug resolution process. In the proposed workflow, PMs retain strategic decision-making power, such as determining whether a bug should be fixed and setting deadlines for the subsequent steps in the bug tracking process. However, they

³³<https://replay.io/>

³⁴<https://www.rrweb.io/>

³⁵<https://www.qawolf.com/>

³⁶<https://traceroo.ai/>

³⁷<https://www.withkeystone.com/>

³⁸<https://www.jazzberry.ai/>

Table 2. Transformation of Roles and Responsibilities of Stakeholders From Traditional Bug Tracking Workflow to Proposed Workflow

Role	Before	Proposed
End-User	Manually reports bugs and confirms fixes.	Uses a chatbot to report bugs and gets instant replies. LLM agents enhance already created bug reports.
Customer Support	Manually classifies, validates, and reproduces bugs. Communicates with developers and end users.	Supervises LLM agents in the HIL system. LLM agents automate bug validity check, classification, reproduction, and no-code fixes.
PM/Team Lead	Manually decides on if bug fixes happen, priority, and deadlines. Assigns bugs to developers.	Decides on bug fixes and deadlines. Reviews bot-assigned priorities and developer assignments.
Developer	Manually locates, reproduces, and fixes bugs.	Reviews and modifies code fixes suggested by LLM agents. LLM agents automate bug reproduction and localization.
Reviewer	Manually reviews and approves/rejects code changes.	Reviews the bug fixes if code is generated by the developer. If code is generated by the agent, the assigned developer performs the primary human review and the reviewer has no responsibility.
Tester	Manually writes and executes test cases. Verifies all fixes.	Supervises verification step in a HIL setup.
Ops Team	Oversees automated deployment and CI/CD pipelines.	Oversees automated deployment with the assistance of LLM agents.

now supervise agent-generated recommendations for priority and developer assignment, ensuring alignment with project goals. Thus, their role shifts toward overseeing and validating an LLM-assisted decision-making process.

Developer: In the traditional settings, developers manually localize and reproduce assigned bugs. After that, they implement code fixes. In the proposed workflow, developers act as supervisors in the HIL patch generation step. LLM agents handle bug reproduction and localization, and suggest code patches. Developers review, refine, and approve these changes, ensuring correctness and maintainability. They remain ultimately responsible for code quality.

Reviewer: Traditionally, reviewers manually inspect code changes, provide feedback, and approve or reject them according to project quality standards. In the proposed workflow, the reviewer's role becomes more selective rather than disappearing. For agent-generated patches, the assigned developer performs the primary human review, refines the patch when needed, and remains accountable for accepting or rejecting it. Reviewers are mainly involved when a fix is manually authored or substantially revised by a developer, or when additional independent review is required by project policy. Thus, the reviewer's responsibility shifts from routine review of every bug fix to focused oversight of developer-authored, escalated, or higher-risk changes.

Tester: In the traditional workflow, testers manually create and execute test cases to verify code fixes. In the proposed workflow, testers act as supervisors in the HIL testing step. Test agents generate test suites, execute them, and verify both code fixes. Testers oversee this process, review generated test cases, and add or refine

tests that exceed the LLMs' current capabilities. Their role shifts from execution to validation and augmentation, effectively becoming "Test Reviewers."

Ops Team: In the traditional setup, the operations team manages automated deployment and oversees CI/CD pipelines. In the proposed workflow, their core responsibilities remain largely unchanged as deployment is already automated. The primary distinction is that the ops team now oversees the deployment process with the assistance of LLM agents, utilizing them for monitoring and anomaly detection while maintaining final authority over the pipeline.

4 Discussion

This section provides a critical synthesis of the proposed agentic framework, exploring its capacity to redefine bug tracking while identifying the multifaceted barriers to its realization. Section 4.1 examines a comprehensive range of future challenges and risks, encompassing technical failure modes like hallucination and cascading errors, ethical and regulatory hurdles such as bias and privacy, and the inherent difficulty of managing non-deterministic bugs. Section 4.2 translates these findings into practical implications for practitioners, detailing how modularity and HIL design can be leveraged to optimize industrial workflows. Finally, Section 4.3 charts a course for future research, identifying a new landscape of inquiries concerning pipeline orchestration and human-agent accountability.

4.1 Future Challenges and Risks

As our proposed AI-driven bug tracking system relies on multiple automated steps powered by LLMs, several challenges and risks must be addressed to ensure its effectiveness and reliability. This section outlines key challenges that need further research and risks that could impact the system's performance.

4.1.1 Accountability Issue. While LLMs can streamline bug tracking tasks from report generation and prioritization to suggesting potential fixes, they also introduce a significant accountability gap absent in traditional systems. The root of this problem lies in the "black-box" nature of LLM decision-making. Because the rationale behind an LLM's outputs is typically opaque, it is often unclear who or what should be held responsible when its recommendations lead to errors. This ambiguity complicates efforts to trace decisions back to a specific cause or actor and undermines the ability to conduct meaningful post-mortems. To address this problem, our approach incorporates a HIL design that explicitly assigns accountability to designated human reviewers, ensuring that final decisions can be audited and responsibility clearly established.

4.1.2 Limited Generalization Across Different Software Projects. A major future challenge in AI-driven bug tracking is ensuring that models generalize well across diverse software ecosystems. The effectiveness of an LLM-based bug tracking system may vary significantly depending on the programming languages, frameworks, and architectures involved. A model trained on one software type may struggle when applied to another, leading to inconsistencies in bug resolution accuracy. To alleviate this limitation, future systems can support cross-project adaptation by dynamically constructing task-specific model context from relevant project artifacts, such as source code, configuration files, documentation, historical bug reports, and execution traces. Instead of relying on static model knowledge or retraining, the framework can assemble only the most relevant project information at runtime and inject it into the model's working context. This allows the system to adapt to new languages, frameworks, and architectures while preserving generalization across heterogeneous software ecosystems.

4.1.3 Bias in LLM Predictions. LLMs and ML models are trained on large-scale datasets that inevitably encode social, cultural, and technical biases as well as systematic knowledge gaps. These biases, when LLMs are used in bug tracking workflows, may propagate into downstream activities such as severity assessment and report classification, potentially causing uneven treatment of bugs across platforms or programming languages [31].

In specialized software domains, where training data is scarce or domain knowledge is highly specific, such biases may become more pronounced. The challenge lies in identifying how these embedded biases influence different stages of bug tracking and understanding their impact on the fairness and reliability of downstream decisions. Such limitations reflect fundamental constraints of model training and data representation, affecting the baseline trustworthiness of individual predictions produced by the system. To mitigate the impact of such biases in future deployments of our framework, continuous bias auditing and calibration mechanisms can be employed to monitor skewed predictions over time. Moreover, placing human oversight at critical decision points, such as bug validation and assignment, provides an additional safeguard against biased or systematically skewed predictions before they propagate through the pipeline.

4.1.4 Factuality Challenges and Hallucination. A major reliability challenge in LLM-powered bug tracking systems arises from two closely related but conceptually distinct failure modes: factuality errors and hallucination. Although these phenomena are often conflated in practice, recent work explicitly distinguishes between them. In particular, Bang et al. [16] characterize hallucination as model outputs that are inconsistent with either the model’s pretraining corpus or the provided input context, while factuality concerns the correctness of generated content with respect to an external ground-truth oracle. As a result, a response may be factually incorrect without being hallucinated if it remains consistent with the model’s internal knowledge, and conversely, a response may be hallucinated even when it happens to be factually correct.

Within automated bug tracking workflows, this distinction becomes operationally important. Factuality challenges arise when agents generate explanations, reproduction steps, environment assumptions, or patch logic that conflict with the actual behavior of the target system, underlying libraries, or current software ecosystem [4]. These errors are particularly problematic in rapidly evolving software domains, where the real-world knowledge landscape changes after model training, causing internal model knowledge to become outdated and increasing the risk of factual failures [68]. Hallucination, by contrast, manifests as the generation of plausible but unsupported code artifacts and behaviors, including fabricated functions, incorrect APIs, and semantically invalid program logic [173]. In a bug tracking context, this may appear as nonexistent configuration settings, invalid execution steps, or misleading code modifications that conflict with the bug report or repository state. Both failure modes pose serious risks in high-stakes tasks such as bug report enhancement, reproduction, and patch generation, where confidently incorrect outputs can mislead downstream automation and human reviewers.

The severity of these risks is reinforced by recent large-scale analyses. Multiple surveys demonstrate that hallucination is not merely a surface-level generation artifact but an inherent limitation of current LLM generation mechanisms, emerging from probabilistic next-token prediction under uncertainty and imperfect contextual grounding [68, 163]. These studies further emphasize that both hallucination and factuality errors systematically undermine trustworthiness in real-world applications requiring correctness and verifiability [4, 69]. Addressing both failure modes is therefore essential for reliable agentic bug tracking.

Prior works propose a range of mitigation strategies. RAG improves factual grounding by anchoring model outputs in external evidence, substantially reducing both factual errors and unsupported fabrications [68, 69]. More recently, Self-Reflective Retrieval-Augmented Generation (SELF-RAG) extends this idea by allowing the model to dynamically decide when retrieval is needed and to critique both retrieved evidence and its own generations through explicit self-reflection tokens, leading to significant improvements in factuality and citation accuracy across diverse tasks [13]. Self-reflection and iterative reasoning mechanisms further improve reliability by enabling models to detect and revise their own errors across multiple passes [72]. Additional techniques, including hallucination-aware fine-tuning, calibration, and auxiliary verification modules, have also demonstrated effectiveness in reducing hallucinated content, particularly in code generation settings [176].

In our framework, these model-level techniques can be complemented by system-level safeguards. Each major pipeline stage could be subject to validation and human oversight, allowing low-confidence or low-quality agent

outputs to be detected early. When such behavior is observed, the pipeline may be halted and responsibility would be transferred to human operators, preventing erroneous outputs from propagating across subsequent stages and preserving the integrity of the overall bug resolution process. This layered mitigation strategy would enable both factuality challenges and hallucination to be addressed through coordinated model improvements and robust system-level governance.

4.1.5 Accumulated Errors Due to Multi-Step LLM Dependency. One of the primary risks in our approach is that multiple stages of the bug lifecycle, such as reporting, reproduction, localization, and patch generation, depend on LLMs, each of which has inherent limitations in accuracy. Since each step builds upon the output of the previous step, poor performance may propagate and amplify, similar to earlier studies [73]. Thus, even small inaccuracies introduced early in the pipeline could compound into major failures at later stages, significantly reducing the final accuracy of bug resolution.

To address this risk, our framework could employ explicit error detection and propagation control mechanisms throughout the pipeline. Each major stage could produce, in addition to its primary output, an associated confidence signal derived from model uncertainty, execution feedback, or downstream validation results.

For example, during bug reproduction, an agent could attempt to execute the synthesized S2R in a sandboxed environment; repeated execution failures, unexpected exceptions, or inconsistent outputs across multiple runs could be interpreted as strong indicators of low confidence in the reproduction result. Similarly, during patch generation and verification, failing test cases, compiler errors, or unstable behavior under repeated execution could automatically lower confidence in the generated patch.

When such confidence falls below predefined thresholds, automated processing could be halted, and the task could be escalated to human operators for inspection and correction. Furthermore, validation checkpoints between stages could prevent unreliable intermediate artifacts, such as incorrect reproduction traces or unstable patches, from propagating further across the workflow. By combining confidence-aware gating, human-in-the-loop escalation, and stage-wise validation, the framework could limit cascading failures while preserving the efficiency benefits of automation.

4.1.6 Agentic Evaluation Challenges. Evaluating LLM/AI Agents is inherently complex [170]. Unlike traditional systems, where well-defined metrics such as accuracy or precision can be applied against a fixed gold standard, an agent-based workflow introduces fluidity that resists simple benchmarking. Many of the tasks performed by the agents lack a clear reference output, making it difficult to establish what constitutes a “correct” answer [78].

This problem has been widely recognized in the literature on agent evaluation. For instance, benchmarks for LLM/AI agents are often fragile, non-standardized, and overfitted to narrow test cases, which prevents them from reflecting real-world utility [78]. Similarly, surveys of LLM-based agents in software engineering stress that many agent tasks are inherently open-ended and multi-modal, making them difficult to evaluate using traditional gold-standard datasets [151]. The AgentBench [97] framework echoes this by highlighting that evaluating agents requires interactive, multi-turn, open-ended environments rather than static correctness checks.

Bug report enhancement exemplifies this challenge clearly. When an agent rewrites or augments a bug report, there is no canonical “ground truth” version of an enhanced report. Multiple plausible enhancements may exist, each potentially useful in practice. In such one-to-many generation tasks, conventional evaluation techniques such as BLEU or ROUGE fail because they assume a single correct output. Indeed, research in text simplification has shown that BLEU often misaligns with human judgments when multiple valid outputs exist, penalizing useful simplifications [130]. More recent work [1] has explored domain-specific metrics such as CTQRS (Crowdsourced Test Report Quality Score), which evaluates bug reports based on structured properties like completeness, clarity, and reproducibility. While such metrics provide a more holistic alternative to surface-level similarity measures, they still rely on predefined rules and cannot fully capture the practical usefulness of an enhanced report in real-world settings.

Bug assignment faces related challenges. Historical assignments could be used as training or evaluation data, but they often contain errors, misallocations, or excessive bug tossing. Using them as a gold standard risks reinforcing suboptimal practices. Moreover, our methodology deliberately differs from past approaches by assigning even invalid bugs to ensure accountability. This introduces a structural misalignment between our workflow and existing datasets: most publicly available datasets focus only on valid bug reports and downstream tasks such as localization or fixing, but they do not include cases of invalid bugs, no-code fixes, or iterative chatbot–user interactions. As a result, these datasets cannot fully capture the scope of our framework, which models the entire bug lifecycle end-to-end [151].

Another important difficulty lies in determining when reproduction or verification has truly failed. In traditional settings, failure is explicit, for instance, when a test case crashes or produces an incorrect output. In agent-based workflows, however, the situation is less clear. An agent may execute a sequence that does not reproduce the bug, but it is ambiguous whether this is because the reproduction was genuinely unsuccessful, because the inputs or environment variables were incomplete, or because the verification criteria were too strict. Similarly, verification may report a fix as successful while subtle side effects or untested conditions still exist. This ambiguity means that even deciding whether an iteration has failed becomes a challenge in itself, complicating both system design and evaluation.

These difficulties highlight a broader methodological gap in evaluating agent-based systems. Unlike deterministic algorithms, agents operate iteratively, refine their outputs, and adapt based on feedback. The quality of such systems cannot be measured only by whether the first output matches a historical reference. Instead, evaluation must consider whether the agent converges toward a useful outcome within practical limits of time and effort. As the literature emphasizes, reproducibility, process-based metrics, and HIL assessments are essential to capturing agent effectiveness [78, 97].

4.1.7 Challenges in Dataset Availability and Reliability. A significant challenge in advancing automated bug tracking systems is the scarcity of high-quality datasets that comprehensively capture real-world bug lifecycles. To reliably test the outcomes of such a system, there is a fundamental need for trusted data sources and benchmarks that can serve as a standard for verifying results. However, publicly available datasets are often incomplete or fragmented, while proprietary systems remain inaccessible due to confidentiality constraints.

Lack of datasets and benchmarks for evaluating bug tracking systems limits quantitative analysis of results, which can reduce reliability.

4.1.8 Privacy Challenges of LLM-Powered Agents. Integrating LLM-powered agents into bug tracking pipelines introduces significant privacy risks. Because these agents process user-submitted bug reports, execution logs, screenshots, and potentially sensitive contextual data, they create new attack surfaces for privacy leakage.

One challenge lies in memory extraction, where adversaries can design prompts to force agents to reveal stored or historical user information. Studies have shown that private memory contents in agent systems can be exfiltrated through black-box interactions, raising concerns about retaining sensitive bug data [141].

A second risk comes from tool-augmented agents, which interact with external systems such as file repositories or CI/CD pipelines. These integrations expand the attack surface, as malicious prompts or poisoned inputs can trick agents into exposing private logs or configuration data [61].

Privacy leakage is also possible through interface manipulation. For example, LLM-powered GUI agents have been shown to be vulnerable to fine-print injection, where invisible or low-salience instructions are embedded in user interfaces, leading the agent to execute unintended actions that compromise sensitive data [29].

Another important challenge is the general opacity of agent behavior makes privacy risks harder to detect. Since agents continuously adapt their responses based on memory and prompts, unintentional disclosure of private bug reports or logs may go unnoticed until after data has been leaked. This unpredictability means that privacy risks are not isolated to adversarial cases but may also arise during normal operations [173]. To reduce

these risks, future AI-powered bug tracking frameworks must carefully control what agents are allowed to store, access, and expose by enforcing strict data-scoping policies, isolating sensitive information across agent components, and incorporating continuous auditing mechanisms that monitor agent interactions for unintended disclosure before privacy breaches propagate.

Beyond technical risks, these challenges also raise important regulatory concerns. In particular, the use of LLM-powered agents in bug tracking systems may conflict with data protection frameworks such as the General Data Protection Regulation (GDPR)³⁹. Bug reports and associated artifacts can contain personally identifiable information (PII), system configurations, or user behavior traces, which fall under GDPR's scope. The continuous collection, storage, and processing of such data by autonomous agents complicate key GDPR requirements, including data minimization, purpose limitation, and the right to erasure. Moreover, the opaque and stateful nature of agent memory makes it difficult to ensure compliance with transparency and accountability obligations. As a result, integrating LLM-based agents into software engineering workflows necessitates not only technical safeguards but also privacy-aware system design aligned with regulatory standards.

4.1.9 Economic Implications and Hidden Costs. Beyond the immediate technical and ethical challenges, the adoption of AI-powered bug tracking systems presents a complex set of economic considerations that go far beyond simple licensing fees. While initial cost-benefit analyses might focus on the efficiency gains from automation, they often fail to account for the substantial hidden costs. These include the financial overhead of API calls, which can escalate unexpectedly with increased usage, as well as the significant infrastructure and computational costs for organizations choosing to host and fine-tune models internally. The cost of integrating these new systems with existing legacy tools and workflows can also be a major expense, requiring extensive development and testing. To mitigate these economic risks, organizations can introduce systematic usage monitoring and cost-aware scheduling mechanisms that regulate how and when AI resources are consumed. By tracking model usage, limiting unnecessary invocations, and prioritizing high-impact operations, such mechanisms help prevent uncontrolled resource growth and ensure that the economic benefits of automation are not offset by hidden operational costs as the system scales.

4.1.10 Complexity of Non-Deterministic and Environmental Faults. While the proposed framework addresses the generation of patches for deterministic software defects, it is crucial to acknowledge the challenges posed by faults that defy straightforward code correction. A significant class of failures falls under the category of *Mandelbugs*, faults characterized by complex activation mechanisms, involving system-internal environmental interactions or propagation delays, resulting in non-deterministic behavior that is difficult to isolate or reproduce [55]. Unlike *Bohrbugs*, which manifest consistently under a defined set of inputs, Mandelbugs often arise from transient conditions such as race conditions, thread scheduling, or specific timing lags, rendering standard reproduction scripts ineffective.

Furthermore, a substantial subset of reported failures does not stem from incorrect source code logic but from environment-dependent and stateful conditions. These include transient issues such as network latency or service timeouts that resolve without intervention, configuration dependencies where bugs manifest only under specific system or database states, and resource constraints related to hardware limitations or memory exhaustion.

In a real example taken from [136], an aging-related Mandelbug causes non-deterministic server crashes in a tax system due to the cumulative exhaustion of the JVM heap during large-scale data processing. The failure probability increases over time as the system state degrades, requiring manual interventions like heap size adjustments and periodic restarts to restore stability.

In another example [136], timing-related Mandelbug occurs in a stock exchange system when a modification request is erroneously rejected because its corresponding order entry is still pending in a secondary processing

³⁹<https://gdpr-info.eu/>

queue. The failure is non-deterministic and load-dependent, manifesting only when specific timing conditions cause the order book update to lag behind incoming modification attempts.

Severe disk exhaustion can lead to physical hardware limitations where the existing storage infrastructure is no longer sufficient to handle the data load. These scenarios require manual physical maintenance, such as the physical installation of new drives or the removal of legacy hardware, to provide the necessary capacity for the system to resume operations.

Addressing these bugs requires a paradigm shift from static code analysis to dynamic system observability. Since the root cause often lies outside the codebase (e.g., in the deployment environment or infrastructure), automated patch generation techniques focused solely on syntax and logic modification are insufficient. Consequently, handling these failures requires not only code repair mechanisms but also operational interventions to address infrastructure and resource constraints. Therefore, the mitigation of Mandelbugs and environmental faults is considered outside the scope of this study, which focuses on the automated repair of deterministic logic defects.

4.1.11 Organizational Resistance, Role Displacement, and Change-Management Risk. A key socio-technical risk of the proposed workflow is organizational resistance driven by perceived role displacement. As summarized in Table 2, several stakeholders’ responsibilities shift from hands-on execution (e.g., manual triage, reproduction, test execution, and operational deployment activities) toward supervision and exception handling in a human-in-the-loop (HIL) setting. While this redesign is intended to reduce TTR and coordination overhead, it can trigger practitioner backlash, fear of job loss, and concerns about deskilling—particularly for roles where automation is explicitly positioned as taking over core activities (e.g., customer support’s validation/reproduction and no-code fixes; testers’ execution-heavy verification tasks; ops’ deployment pipeline work).

This risk is not only a cultural adoption issue; it can directly impact system performance. If practitioners distrust automated outputs or perceive the system as replacing rather than augmenting them, they may (i) resist using the workflow, (ii) bypass automation (reverting to “shadow processes” outside the tracker), (iii) provide minimal feedback needed for continuous improvement, or (iv) over-correct by adding excessive manual checkpoints—negating the intended efficiency gains. This concern complements the broader set of challenges already discussed under “Future Challenges and Risks,” which currently emphasize technical and governance limitations (e.g., cascading errors, accountability, evaluation, privacy, and cost).

Mitigating this challenge requires explicit change-management and job-redesign strategies alongside technical design. Concretely, future implementations should: (1) define “automation boundaries” per role (what is automated vs. what remains human-owned) and communicate them transparently; (2) frame the new responsibilities as capability uplift (e.g., “test reviewer,” “support supervisor,” “ops governor”) and invest in reskilling paths aligned with those responsibilities; (3) adopt phased rollouts and pilots where automation starts as decision support before becoming default, with measurable opt-out criteria; and (4) ensure the human-in-the-loop (HIL) checkpoints make people’s work easier, not harder, by clearly explaining why the system made a recommendation and by specifying when a person should step in or escalate the issue, so responsibility stays clear and manageable.

4.2 Implications for Practitioners

The proposed framework is not only a technical contribution but also a practical blueprint for how modern software teams can restructure their bug management processes in the presence of intelligent automation. Its successful adoption depends on organizational context, team structure, existing development practices, and the degree to which human oversight is integrated into automated workflows. The following implications highlight key considerations for practitioners, focusing on how the system can be adapted, customized, and integrated into real-world development environments to maximize its impact on productivity, reliability, and software quality.

4.2.1 Adapting to Diverse Project Structures. Our proposed bug tracking system assumes a certain team structure with dedicated roles for PMs, developers, and other team members. However, we acknowledge that this may not align with every project. The organizational structure of software development teams can vary significantly across different projects and companies. It is important for practitioners to adapt our system to fit their specific needs.

For example, a small startup or open source project might not have a separate customer support team, with developers or the PM directly handling bug reports from users. In these cases, the system's bug reporting and triage processes would be streamlined to route information directly to the person who needs to act on it, bypassing a formal customer support or tester role. Similarly, in a small team, a developer might also serve as the tester, responsible for both writing code and creating test cases. The system should be flexible enough to accommodate this by allowing a single user to perform multiple functions, such as creating a bug report, localizing the bug, and implementing the fix.

By being aware of these potential differences, project teams can configure our system to reflect their actual workflow, ensuring that the benefits of automated bug localization and management are realized without forcing a rigid, and potentially inefficient, organizational model upon them. The system's strength lies in its ability to provide clear, context-aware insights, regardless of who is performing the different roles.

4.2.2 Human-in-the-Loop System Training and Customization. The successful adoption of our semi-automated bug tracking framework depends on its ability to integrate seamlessly with existing workflows. A key implication for practitioners is that this is not a "plug-and-play" solution; it is a HIL system that requires training and customization to achieve its full potential.

Our system's AI components, particularly the LLMs for bug localization and patch generation, can be fine-tuned on an organization's proprietary codebase and historical bug data. This process allows the models to learn project-specific nuances, coding conventions, and common bug patterns, making their suggestions more accurate and relevant. This is a crucial step that distinguishes a generic tool from a highly effective, tailored solution.

Beyond this technical fine-tuning, companies must create their own workflows to leverage the system's capabilities. For example, they should define clear rule sets that establish when a developer should accept an AI-generated fix versus when they should manually intervene. They must also establish feedback mechanisms for developers to provide input on the AI's suggestions, allowing the system to continuously learn and improve. Finally, companies should adapt the roles and responsibilities of their teams to maximize the benefits of automation; with the system handling routine tasks, developers can focus on high-level design and complex bugs, while testers can concentrate on validating the AI's fixes and exploring edge cases. By actively engaging in this HIL training and custom adoption, practitioners can ensure the system becomes an integrated and invaluable part of their development process, maximizing productivity and code quality.

4.2.3 Modular System Architecture and Process Optimization. The components of our bug tracking system are designed with a modular architecture, meaning each part, from bug report enhancement and reproduction to fault localization and patch generation, can be run either independently or as part of a single, unified workflow. This is a critical implication for practitioners, as it allows for significant flexibility and process optimization based on a project's unique needs and constraints.

A project with a mature CI/CD pipeline, for example, might choose to run the entire system end-to-end, from the moment a bug is reported to the automated creation of a pull request with a suggested fix. In contrast, a team working on a legacy system or in a more manual environment might opt to use only a single, specific component. They could, for instance, utilize the localization feature as a standalone tool to rapidly pinpoint the buggy code, then rely on human developers to manually create and test the fix. Similarly, a project with a dedicated QA team might choose to use the AI for bug report enhancement and reproduction, allowing testers to become more efficient without fully automating the fix generation step.

This modularity empowers development teams to choose a level of automation that suits their specific context, minimizing disruption while maximizing the system's benefits. By allowing projects to selectively adopt and integrate the most valuable components, our system becomes a highly adaptable tool that can be optimized for a wide range of software development processes.

4.2.4 System Integration and Toolchain Adapters. Successfully integrating such a bug tracking toolchain into existing systems requires a strategic approach focused on building flexible adapters and connectors. These components serve as a crucial layer, allowing the system to communicate with and leverage existing development tools without requiring a complete overhaul of an organization's infrastructure. It is essential to develop these connectors for critical parts of the toolchain, such as VCSs like Git, which would allow the system to pull code and submit AI-generated patches as pull requests. Similarly, adapters for integrated development environments like Visual Studio Code⁴⁰ or IntelliJ⁴¹ would provide developers with real-time AI recommendations and debugging insights directly within their coding environment. Furthermore, building connectors for CI/CD platforms such as Jenkins⁴² or GitHub Actions is vital for automating the testing and validation of AI-generated fixes, ensuring that only verified code is merged. Finally, adapters for pre-existing bug tracking systems like Jira would allow for the seamless import of bug reports, enabling a phased and non-disruptive adoption. This approach ensures that the new system acts as a powerful enhancement to the existing ecosystem, maximizing the return on the integration effort.

4.3 Implications for Researchers

The proposed framework highlights not only the technical contributions of automation but also opens a broad set of research opportunities. These implications arise at multiple levels, including the ordering of activities within the workflow, the role and positioning of human oversight, the scope of individual agent capabilities, and the ways in which domain- or bug-type specific factors shape system effectiveness. To structure these implications, we organize the discussion into several thematic areas, each representing a distinct line of inquiry for future research.

4.3.1 Ordering of Activities. Our methodology introduces automation across the bug lifecycle, but it also raises questions about how these activities should be sequenced. In our design, bug report enhancement precedes reproduction, which in turn is followed by classification, assignment, localization, and patch generation. While this ordering reflects common practice, it is not inherently optimal. For instance, running classification before reproduction could help filter out low-priority reports and save computational resources, while prioritizing reproduction ensures that only actionable reports move downstream. These trade-offs point to a larger research opportunity: systematically studying how different pipeline orderings influence accuracy, efficiency, and TTR.

4.3.2 Human Oversight and Accountability. Automation cannot fully replace human judgment, and the positioning of HIL interventions is itself an open research question. For example, requiring project manager or team lead approval before assignment may ensure correctness but add delays, while placing oversight after assignment may reduce turnaround but risk misallocation. As another example, if a user submits a trivial or well-known bug, such as a minor UI inconsistency, a typo in the interface, or a frequently encountered issue that customer support is already familiar with, the agents may still attempt reproduction and classification. However, in such cases, customer support could immediately recognize the problem and provide a direct resolution without going through the entire automated pipeline. Similar trade-offs exist for reviewers, testers, and customer support. Studying these interactions will clarify how to balance automation with accountability in agent-based bug tracking systems.

⁴⁰<https://code.visualstudio.com/>

⁴¹<https://www.jetbrains.com/idea/>

⁴²<https://www.jenkins.io/>

4.3.3 Agent Capabilities as Research Problems. As illustrated in Figure 2, each capability represented in the workflow, such as bug report enhancement, reproduction, classification, localization, patch generation, and verification, can itself be viewed as a distinct research problem. Advances in NLP and LLMs can refine bug report enhancement, program analysis techniques can strengthen localization, and test generation research can improve reproduction and verification. Researchers can therefore explore each of the boxes in Figure 2 as independent research topics, while also examining how these functions interoperate within an end-to-end framework that spans the entire bug tracking system.

4.3.4 Domain-Specific Challenges. The effectiveness of automated bug tracking will vary across domains. For example, in on-demand delivery systems, bugs often occur immediately, making them straightforward to detect and reproduce. In contrast, interactive environments such as gaming may require long execution traces or rare conditions before a bug surfaces [167], complicating both reproduction and verification. These contrasts suggest that domain-specific adaptations of automation strategies are a promising research direction.

4.3.5 Bug Type Differences. Not all bug types are equal in terms of detection and automation. Crash bugs are often captured directly through logs or monitoring, making them easier to handle in automated workflows. Functional, performance, or usability bugs, however, are more nuanced and may require user modeling, advanced instrumentation, or long-term observation. Future research can investigate which automation strategies are most effective for which bug types, and how to design adaptive systems that tailor their methods accordingly.

Taken together, these considerations show that automated bug tracking systems should not be viewed as a single research challenge but rather as an ecosystem of interrelated problems. Investigating variations in task ordering, agent capabilities, human oversight, domain-specific challenges, and bug type detection can significantly advance our understanding of how intelligent bug tracking systems should be designed and deployed.

5 Conclusion

We presented a vision for a next-generation bug tracking framework that leverages LLMs in each possible step of bug tracking. By incorporating LLM-driven automation into report creation and enhancement, reproduction, classification, validation, localization, resolution, and verification, the proposed system aims to reduce developer workload and decrease TTR while improving the quality and efficiency of bug tracking and software maintenance in general.

The proposed system extends traditional bug tracking systems by employing conversational assistant agents to support users in creating bug reports, and later enhancing possibly overlooked fields in the bug report, automating the execution of reproduction steps, and improving the accuracy of bug classification and validation. Furthermore, the integration of no-code fix suggestions and LLM-assisted bug localization, followed by patches generated and deployed by LLM agents, provides the potential to decrease resolution times and optimize resource allocation.

Despite its potential to significantly reduce time-to-resolution and coordination overhead, AI-driven bug tracking introduces a complex set of interrelated technical, methodological, and socio-organizational challenges. These include accountability gaps stemming from opaque LLM decision-making, limited cross-project generalization, embedded biases in model predictions, and critical reliability risks such as factuality errors, hallucinations, and cascading failures in multi-step pipelines. Beyond model behavior, fundamental challenges arise in evaluating agentic systems due to the absence of clear ground truth, as well as the scarcity and unreliability of comprehensive bug lifecycle datasets. Additional concerns include privacy and regulatory risks associated with sensitive data handling, non-trivial economic costs related to infrastructure and scaling, and the inability of current approaches to address non-deterministic and environment-dependent faults. Finally, successful adoption depends not only on technical robustness but also on effective change management, as shifts toward human-in-the-loop workflows

may trigger organizational resistance, role redefinition challenges, and potential degradation in practitioner trust and system utilization.

In conclusion, this work provides a conceptual framework for an adaptive, LLM-driven bug tracking ecosystem. Future research is required to operationalize these proposals, empirically evaluate their effectiveness in real-world software projects, and further refine the underlying models and methodologies.

6 Acknowledgements

This work was supported by TÜBİTAK (The Scientific and Technological Research Council of Turkey) under the 1001 Scientific and Technological Research Projects Funding Program, Project No. 125E371. The authors gratefully acknowledge TÜBİTAK for its support.

Just Accepted

References

- [1] Jagrit Acharya and Gouri Ginde. 2025. Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation. *arXiv preprint arXiv:2504.18804* (2025).
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Emre Furkan Akyol, Mehmet Dedeler, and Eray Tüzün. 2026. ImproBR: Bug Report Improver Agent Using LLMs. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '26)*. ACM.
- [4] Aisha Alansari and Hamzah Luqman. 2025. Large Language Models Hallucination: A Comprehensive Survey. *arXiv preprint arXiv:2510.06265* (2025).
- [5] Syed Juned Ali, Varun Naganathan, and Dominik Bork. 2024. Establishing traceability between natural language requirements and software artifacts by combining rag and LLMs. In *International Conference on Conceptual Modeling*. Springer, 295–314.
- [6] Ebube Alor, SayedHassan Khatoonabadi, and Emad Shihab. 2025. Evaluating the Use of LLMs for Documentation to Code Traceability. *arXiv preprint arXiv:2506.16440* (2025).
- [7] Uğur Can Altun, Ismail Seren Göçmen, Emre Sülün, Erdem Tuna, and Eray Tüzün. 2025. Process smells in practice: an evaluative case study. *Empirical Software Engineering* 30, 5 (2025), 115.
- [8] Prasanth Anbalagan and Mladen Vouk. 2009. On predicting the time taken to correct bug reports in open source projects. In *2009 IEEE International Conference on Software Maintenance*. IEEE, 523–526.
- [9] John Anvik, Lyndon Hiew, and Gail C Murphy. 2006. Who should fix this bug?. In *Proceedings of the 28th international conference on Software engineering*. 361–370.
- [10] Leonhard Applis, Yuntong Zhang, Shanchao Liang, Nan Jiang, Lin Tan, and Abhik Roychoudhury. 2025. Unified Software Engineering agent as AI Software Engineer. *arXiv preprint arXiv:2506.14683* (2025).
- [11] P Ardimento, Michele Capuzzimati, Gabriella Casalino, Daniele Schicchi, and Davide Taibi. 2025. A novel LLM-based classifier for predicting bug-fixing time in Bug Tracking Systems. *Journal of Systems and Software* (2025), 112569.
- [12] Robert S Arnold. 1996. *Software change impact analysis*. IEEE Computer Society Press.
- [13] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. (2024).
- [14] Mona Attariyan and Jason Flinn. 2010. Automating configuration troubleshooting with dynamic information flow analysis. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*.
- [15] Thazin Win Win Aung, Huan Huo, and Yulei Sui. 2020. A literature review of automatic traceability links recovery for software change impact analysis. In *Proceedings of the 28th International Conference on Program Comprehension*. 14–24.
- [16] Yejin Bang, Ziwei Ji, Alan Schelten, Anthony Hartshorn, Tara Fowler, Cheng Zhang, Nicola Cancedda, and Pascale Fung. 2025. Hallulens: Llm hallucination benchmark. *arXiv preprint arXiv:2504.17550* (2025).
- [17] Olga Baysal, Michael W. Godfrey, and Robin Cohen. 2009. A bug you like: A framework for automated assignment of bugs. In *2009 IEEE 17th International Conference on Program Comprehension*. 297–298. doi:10.1109/ICPC.2009.5090066
- [18] Dane Bertram, Amy Voida, Saul Greenberg, and Robert Walker. 2010. Communication, collaboration, and bugs: the social nature of issue tracking in small, colocated teams. In *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work*. 291–300.
- [19] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiß, Rahul Premraj, and Thomas Zimmermann. 2007. Quality of bug reports in eclipse. In *Proceedings of the 2007 OOPSLA workshop on eclipse technology eXchange*. 21–25.
- [20] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What makes a good bug report?. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*. 308–318.
- [21] Pamela Bhattacharya, Iulian Neamtii, and Christian R. Shelton. 2012. Automated, highly-accurate, bug assignment using machine learning and tossing graphs. *Journal of Systems and Software* 85, 10 (2012), 2275–2292. doi:10.1016/j.jss.2012.04.053 Automated Software Evolution.
- [22] Christian Bird, Nachiappan Nagappan, Premkumar Devanbu, Harald Gall, and Brendan Murphy. 2009. Does distributed development affect software quality? an empirical case study of windows vista. *Commun. ACM* 52, 8 (2009), 85–93.
- [23] Lili Bo, Wangjie Ji, Xiaobing Sun, Ting Zhang, Xiaoxue Wu, and Ying Wei. 2024. ChatBR: Automated assessment and improvement of bug report quality using ChatGPT. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1472–1483.
- [24] Silvia Breu, Rahul Premraj, Jonathan Sillito, and Thomas Zimmermann. 2009. Frequently asked questions in bug reports. (2009).
- [25] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the quality of the steps to reproduce in bug reports. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 86–96.
- [26] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of the 2017 11th joint meeting on foundations of software*

- engineering. 396–407.
- [27] Krishna Kumar Chaturvedi and VB Singh. 2012. Determining bug severity using machine learning techniques. In *2012 CSI sixth International Conference on Software Engineering (CONSEG)*. IEEE, 1–6.
 - [28] Bo Chen, Zhenkun Yang, Li Lei, Kai Cong, and Fei Xie. 2020. Automated bug detection and replay for COTS linux kernel modules with concolic execution. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 172–183.
 - [29] Chaoran Chen, Zhiping Zhang, Bingcan Guo, Shang Ma, Ibrahim Khalilov, Simret A Gebreegziabher, Yanfang Ye, Ziang Xiao, Yaxing Yao, Tianshi Li, et al. 2025. The Obvious Invisible Threat: LLM-Powered GUI Agents' Vulnerability to Fine-Print Injections. *arXiv preprint arXiv:2504.11281* (2025).
 - [30] Lianping Chen. 2015. Continuous Delivery: Huge Benefits, but Challenges Too. *IEEE Software* 32, 2 (2015), 50–54. doi:10.1109/MS.2015.27
 - [31] QiHong Chen, Jiachen Yu, Jiawei Li, Jiecheng Deng, Justin Tian Jin Chen, and Iftekhar Ahmed. 2024. A Deep Dive Into Large Language Model Code Generation Mistakes: What and Why? *arXiv preprint arXiv:2411.01414* (2024).
 - [32] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic bug reproduction for effective automated program repair at google. *arXiv preprint arXiv:2502.01821* (2025).
 - [33] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
 - [34] Davor Cubranic and Gail C. Murphy. 2004. Automatic bug triage using text categorization. In *International Conference on Software Engineering and Knowledge Engineering*. <https://api.semanticscholar.org/CorpusID:16196403>
 - [35] Michael A Cusumano and David B Yoffie. 1999. Software development on Internet time. *Computer* 32, 10 (1999), 60–69.
 - [36] Anca Deak, Tor Stålhane, and Guttorm Sindre. 2016. Challenges and strategies for motivating software testing personnel. *Information and software Technology* 73 (2016), 1–15.
 - [37] Václav Dedík and Bruno Rossi. 2016. Automated Bug Triaging in an Industrial Context. In *2016 42th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 363–367. doi:10.1109/SEAA.2016.20
 - [38] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805 [cs.CL]* <https://arxiv.org/abs/1810.04805>
 - [39] Emre Dinç and Eray Tüzün. 2025. Judge the Votes: A System to Classify Bug Reports and Give Suggestions. In *Proceedings of the 2nd ACM International Conference on AI-powered Software (Alware)*.
 - [40] Tung Do Viet and Konstantin Markov. 2023. Using large language models for bug localization and fixing. In *2023 12th International Conference on Awareness Science and Technology (iCAST)*. IEEE, 192–197.
 - [41] Xiaoting Du, Zhihao Liu, Chenglong Li, Xiangyue Ma, Yingzhuo Li, and Xinyu Wang. 2024. LLM-BRC: A large language model-based bug report classification framework. *Software Quality Journal* 32, 3 (2024), 985–1005.
 - [42] Çağrı Eren, Kerem Şahin, and Eray Tüzün. 2023. Analyzing bug life cycles to derive practical insights. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. 162–171.
 - [43] Mona Erfani Joorabchi, Mehdi Mirzaaghaei, and Ali Mesbah. 2014. Works for me! characterizing non-reproducible bug reports. In *Proceedings of the 11th Working Conference on Mining Software Repositories*. 62–71.
 - [44] Yuanrui Fan, Xin Xia, David Lo, and Ahmed E Hassan. 2018. Chaff from the wheat: Characterizing and determining valid bug reports. *IEEE transactions on software engineering* 46, 5 (2018), 495–525.
 - [45] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
 - [46] Dominik Fuchß, Haoyu Liu, Tobias Hey, Jan Keim, and Anne Koziolk. 2025. Enabling architecture traceability by LLM-based architecture component name extraction. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 1–12.
 - [47] Guilherme Gadelha, Franklin Ramalho, and Tiago Massoni. 2021. Traceability recovery between bug reports and test cases-a Mozilla Firefox case study. *Automated Software Engineering* 28, 2 (2021), 8.
 - [48] Ali Ghanbari, Deepak-George Thomas, Muhammad Arbab Arshad, and Hridayesh Rajan. 2023. Mutation-based fault localization of deep neural networks. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1301–1313.
 - [49] Debolina Ghosh and Jagannath Singh. 2019. A systematic review on program debugging techniques. *Smart Computing Paradigms: New Progresses and Challenges: Proceedings of ICACNI 2018, Volume 2* (2019), 193–199.
 - [50] Ismail Seren Göçmen, Ahmed Salih Cezayir, and Eray Tüzün. 2025. Enhanced code reviews using pull request based change impact analysis. *Empirical Software Engineering* 30, 3 (2025), 64.
 - [51] Orlena CZ Gotel and CW Finkelstein. 1994. An analysis of the requirements traceability problem. In *Proceedings of IEEE International Conference on Requirements Engineering*. IEEE, 94–101.
 - [52] Georgios Gousios, Martin Pinzger, and Arie van Deursen. 2014. An exploratory study of the pull-based software development model. In *Proceedings of the 36th International Conference on Software Engineering*. 345–355.

- [53] Anjali Goyal and Neetu Sardana. 2019. An empirical study of non-reproducible bugs. *International Journal of System Assurance Engineering and Management* 10, 5 (2019), 1186–1220.
- [54] Jim Gray. 1986. Why do computers stop and what can be done about it?. In *Symposium on reliability in distributed software and database systems*. Los Angeles, CA, USA, 3–12.
- [55] Michael Grottke and Kishor S Trivedi. 2005. A classification of software faults. *Journal of Reliability Engineering Association of Japan* 27, 7 (2005), 425–438.
- [56] Jin LC Guo, Jan-Philipp Steghöfer, Andreas Vogelsang, and Jane Cleland-Huang. 2025. Natural language processing for requirements traceability. In *Handbook on Natural Language Processing for Requirements Engineering*. Springer, 89–116.
- [57] Philip J Guo, Thomas Zimmermann, Nachiappan Nagappan, and Brendan Murphy. 2010. Characterizing and predicting which bugs get fixed: an empirical study of microsoft windows. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1*. 495–504.
- [58] Ahmed E Hassan. 2009. Predicting faults using the complexity of code changes. In *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 78–88.
- [59] Ahmed E. Hassan, Dayi Lin, Gopi Krishnan Rajbahadur, Keheliya Gallaba, Filipe R. Cogo, Boyuan Chen, Haoxiang Zhang, Kishanthan Thangarajah, Gustavo Ansal di Oliva, Jiahuei Lin, Wali Mohammad Abdullah, and Zhen Ming Jiang. 2024. Rethinking Software Engineering in the Foundation Model Era: A Curated Catalogue of Challenges in the Development of Trustworthy FMware. arXiv:2402.15943 [cs.SE] <https://arxiv.org/abs/2402.15943>
- [60] Jameleddine Hassine. 2024. An LLM-based Approach to Recover Traceability Links between Security Requirements and Goal Models. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (Salerno, Italy) (EASE '24)*. Association for Computing Machinery, New York, NY, USA, 643–651. doi:10.1145/3661167.3661261
- [61] Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S Yu. 2024. The emerged security and privacy of LLM agent: A survey with case studies. *arXiv preprint arXiv:2407.19354* (2024).
- [62] Jianjun He, Ling Xu, Yuanrui Fan, Zhou Xu, Meng Yan, and Yan Lei. 2020. Deep learning based valid bug reports determination and explanation. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 184–194.
- [63] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. *International Conference on Learning Representations, ICLR*.
- [64] John W Horch. 2003. *Practical guide to software quality management*. Artech House.
- [65] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [66] Jeff Huang and Charles Zhang. 2016. Debugging concurrent software: Advances and challenges. *Journal of Computer Science and Technology* 31, 5 (2016), 861–868.
- [67] Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. 2024. Evolving paradigms in automated program repair: Taxonomy, challenges, and opportunities. *Comput. Surveys* 57, 2 (2024), 1–43.
- [68] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [69] Yue Huang, Lichao Sun, Haoran Wang, Siyuan Wu, Qihui Zhang, Yuan Li, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, et al. 2024. Trustllm: Trustworthiness in large language models. *arXiv preprint arXiv:2401.05561* (2024).
- [70] Mia Mohammad Imran, Agnieszka Ciborowska, and Kostadin Damevski. 2021. Automatically Selecting Follow-up Questions for Deficient Bug Reports. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 167–178. doi:10.1109/MSR52588.2021.00029
- [71] Mustafa Özkan İr, Mehmet Dedeler, Anıl Koyuncu, and Eray Tüzün. 2026. Fixpad++: Automated Bug Fix Verification Using LLM Agents. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AI-Ware '26)*. ACM.
- [72] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
- [73] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [74] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1424–1446.
- [75] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering* 30, 2 (2025), 45.
- [76] Sungmin Kang, Juyeon Yoon, Nargiz Askarbekkyzy, and Shin Yoo. 2024. Evaluating diverse large language models for automatic and general bug reproduction. *IEEE Transactions on Software Engineering* (2024).

- [77] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
- [78] Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. *arXiv:2407.01502 [cs.LG]* <https://arxiv.org/abs/2407.01502>
- [79] Akhilesh Babu Kolluri, K Tameezuddin, and Kalpana Gudikandula. 2012. Effective Bug Tracking Systems: Theories and Implementation. *IOSR Journal of Computer Engineering (IOSRJCE)*. ISSN (2012), 2278–0661.
- [80] Mikko Korkala, Pekka Abrahamsson, and Pekka Kyllonen. 2006. A case study on the impact of customer communication on defects in agile software development. In *AGILE 2006 (AGILE'06)*. IEEE, 11–pp.
- [81] Anil Koyuncu. 2025. Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* (2025).
- [82] Naveen Krishnan. 2025. Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications. *arXiv preprint arXiv:2504.21030* (2025).
- [83] Ashima Kukkar, Yugal Kumar, Ashutosh Sharma, and Jasminder Kaur Sandhu. 2023. Bug severity classification in software using ant colony optimization based feature weighting technique. *Expert Systems with Applications* 230 (2023), 120573.
- [84] Ashima Kukkar, Rajni Mohana, and Yugal Kumar. 2020. Does bug report summarization help in enhancing the accuracy of bug severity classification? *Procedia Computer Science* 167 (2020), 1345–1353.
- [85] Ashima Kukkar, Rajni Mohana, Anand Nayyar, Jeamin Kim, Byeong-Gwon Kang, and Naveen Chilamkurti. 2019. A novel deep-learning-based bug severity classification technique using convolutional neural networks and random forest with boosting. *Sensors* 19, 13 (2019), 2964.
- [86] Atish Kumar Dipongkor. 2024. An Ensemble Method for Bug Triaging using Large Language Models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (Lisbon, Portugal) (ICSE-Companion '24)*. Association for Computing Machinery, New York, NY, USA, 438–440. doi:10.1145/3639478.3641228
- [87] Muhammad Laiq, Nauman bin Ali, Jürgen Börstler, and Emelie Engström. 2024. Industrial adoption of machine learning techniques for early identification of invalid bug reports. *Empirical Software Engineering* 29, 5 (2024), 130.
- [88] Muhammad Laiq, Nauman bin Ali, Jürgen Böstler, and Emelie Engström. 2022. Early identification of invalid bug reports in industrial settings—a case study. In *International Conference on Product-Focused Software Process Improvement*. Springer, 497–507.
- [89] Ahmed Lamkanfi, Serge Demeyer, Emanuel Giger, and Bart Goethals. 2010. Predicting the severity of a reported bug. In *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. IEEE, 1–10.
- [90] Thanh Le-Cong, Duc-Minh Luong, Xuan Bach D Le, David Lo, Nhat-Hoa Tran, Bui Quang-Huy, and Quyet-Thang Huynh. 2023. Invalidator: Automated patch correctness assessment via semantic and syntactic reasoning. *IEEE Transactions on Software Engineering* 49, 6 (2023), 3411–3429.
- [91] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering* 38, 1 (2011), 54–72.
- [92] Jaehyung Lee, Kisun Han, and Hwanjo Yu. 2023. A Light Bug Triage Framework for Applying Large Pre-trained Language Model. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 3, 11 pages. doi:10.1145/3551349.3556898
- [93] Chenglong Li, Zheng Zheng, Xiaoting Du, Xiangyue Ma, Zhengqi Wang, and Xinheng Li. 2024. KnowBug: Enhancing Large language models with bug report knowledge for deep learning framework bug prediction. *Knowledge-Based Systems* 305 (2024), 112588.
- [94] Kefan Li and Yuan Yuan. 2024. Large language models as test case generators: Performance evaluation and enhancement. *arXiv preprint arXiv:2404.13340* (2024).
- [95] S Li, J Guo, M Fan, JG Lou, Q Zheng, and T Liu. [n. d.]. Automated bug reproduction from user reviews for android applications. 2020 IEEE. In *ACM 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 51–60.
- [96] Yue Li, Bohan Liu, Ting Zhang, Zhiqi Wang, David Lo, Lanxin Yang, Jun Lyu, and He Zhang. 2025. A Knowledge Enhanced Large Language Model for Bug Localization. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1914–1936.
- [97] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2023. AgentBench: Evaluating LLMs as Agents. *arXiv:2308.03688 [cs.AI]* <https://arxiv.org/abs/2308.03688>
- [98] Zhe Liu, Cheng Li, Chunyang Chen, Junjie Wang, Boyu Wu, Yawen Wang, Jun Hu, and Qing Wang. 2024. Vision-driven automated mobile gui testing via multimodal large language model. *arXiv preprint arXiv:2407.03037* (2024).
- [99] Fan Long and Martin Rinard. 2016. Automatic patch generation by learning correct code. In *Proceedings of the 43rd annual ACM SIGPLAN-SIGACT symposium on principles of programming languages*. 298–312.
- [100] Guoming Long, Jingzhi Gong, Hui Fang, and Tao Chen. 2025. Learning Software Bug Reports: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* (July 2025). doi:10.1145/3750040 Just Accepted.
- [101] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*. 643–653.

- [102] Michael R Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamon Thongtanunam. 2024. Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology* (2024).
- [103] Yijing Lyu, Heetae Cho, Pilsu Jung, and Seonah Lee. 2023. A systematic literature review of issue-based requirement traceability. *Ieee Access* 11 (2023), 13334–13348.
- [104] Junayed Mahmud, Antu Saha, Oscar Chaparro, Kevin Moran, and Andrian Marcus. 2025. Combining Language and App UI Analysis for the Automated Assessment of Bug Reproduction Steps. *arXiv preprint arXiv:2502.04251* (2025).
- [105] Ehsan Mashhadi, Hossein Ahmadvand, and Hadi Hemmati. 2023. Method-level bug severity prediction using source code metrics and LLMs. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 635–646.
- [106] Qianru Meng and Joost Visser. 2023. Which bug reports are valid and why? Using the BERT transformer to classify bug reports and explain their validity. In *Proceedings of the 4th European Symposium on Software Engineering*. 52–60.
- [107] Audris Mockus, Roy T Fielding, and James Herbsleb. 2000. A case study of open source software development: the Apache server. In *Proceedings of the 22nd International Conference on Software Engineering*. 263–272.
- [108] Audris Mockus, Roy T Fielding, and James D Herbsleb. 2002. Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 11, 3 (2002), 309–346.
- [109] Simo Mäkinen, Marko Leppänen, Terhi Kilamo, Anna-Liisa Mattila, Eero Laukkanen, Max Pagels, and Tomi Männistö. 2016. Improving the delivery cycle: A multiple-case study of the toolchains in Finnish software intensive enterprises. *Information and Software Technology* 80 (2016), 175–194. doi:10.1016/j.infsof.2016.09.001
- [110] Yu Nong, Haoran Yang, Long Cheng, Hongxin Hu, and Haipeng Cai. 2024. Automated software vulnerability patching using large language models. *arXiv preprint arXiv:2408.13597* (2024).
- [111] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.
- [112] Avinash Patil and Aryan Jadon. 2024. Next-Generation Bug Reporting: Enhancing Development with AI Automation. *Authorea Preprints* (2024).
- [113] Matthias Pohl, Ali Hashaam, Sascha Bosse, Daniel Gunnar Staegemann, Matthias Volk, Frederik Kramer, and Klaus Turowski. 2020. Application of NLP to determine the state of issues in bug tracking systems. In *2020 International Conference on Data Mining Workshops (ICDMW)*. IEEE, 53–61.
- [114] Roger S Pressman. 2005. *Software engineering: a practitioner’s approach*. Palgrave macmillan.
- [115] Khushbakht Ali Qamar, Emre Sülün, and Eray Tüzün. 2022. Taxonomy of bug tracking process smells: Perceptions of practitioners and an empirical analysis. *Information and Software Technology* 150 (2022), 106972.
- [116] Yuhua Qi, Xiaoguang Mao, Yan Lei, Ziyang Dai, and Chengsong Wang. 2014. The strength of random search on automated program repair. In *Proceedings of the 36th International Conference on Software Engineering*. 254–265.
- [117] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2025. Soap FL: A Standard Operating Procedure for LLM-based Method-Level Fault Localization. *IEEE Transactions on Software Engineering* (2025).
- [118] Michael Rath, Jacob Rendall, Jin LC Guo, Jane Cleland-Huang, and Patrick Mäder. 2018. Traceability in the wild: automatically augmenting incomplete trace links. In *Proceedings of the 40th International Conference on Software Engineering*. 834–845.
- [119] Hai Ruan, Binyu Chen, Xin Peng, and Wen Zhao. 2019. DeepLink: Recovering issue-commit links based on deep learning. *Journal of Systems and Software* 158 (2019), 110406.
- [120] Mehil B Shah, Mohammad Masudur Rahman, and Foutse Khomh. 2025. Towards enhancing the reproducibility of deep learning bugs: an empirical study. *Empirical Software Engineering* 30, 1 (2025), 23.
- [121] Lin Shi, Fangwen Mu, Yumin Zhang, Ye Yang, Junjie Chen, Xiao Chen, Hanzhi Jiang, Ziyong Jiang, and Qing Wang. 2022. BugListener: identifying and synthesizing bug reports from collaborative live chats. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE ’22)*. Association for Computing Machinery, New York, NY, USA, 299–311. doi:10.1145/3510003.3510108
- [122] Smithsonian National Museum of American History. 1947. Log Book: Computer Bug (Harvard Mark II, 1947). https://www.si.edu/object/log-book-computer-bug%3Anmah_334663. Accessed: 2025-09-15.
- [123] Mozhan Soltani, Felienne Hermans, and Thomas Bäck. 2020. The significance of bug report elements. *Empirical Software Engineering* 25, 6 (2020), 5255–5294.
- [124] Yang Song and Oscar Chaparro. 2020. Bee: A tool for structuring and analyzing bug reports. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 1551–1555.
- [125] Yang Song, Junayed Mahmud, Nadeeshan De Silva, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2023. Burt: A chatbot for interactive bug reporting. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 170–174.
- [126] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward interactive bug reporting for (android app) end-users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and*

- Symposium on the Foundations of Software Engineering* (Singapore, Singapore) (ESEC/FSE 2022). Association for Computing Machinery, New York, NY, USA, 344–356. doi:10.1145/3540250.3549131
- [127] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation* 28, 1 (1972), 11–21.
 - [128] Brian L Stuart. 2018. Debugging the ENIAC [Scanning Our Past]. *Proc. IEEE* 106, 12 (2018), 2331–2345.
 - [129] Ya-Yunn Su, Mona Attariyan, and Jason Flinn. 2007. Autobash: improving configuration management with operating system causality analysis. *ACM SIGOPS Operating Systems Review* 41, 6 (2007), 237–250.
 - [130] Elior Sulem, Omri Abend, and Ari Rappoport. 2018. BLEU is Not Suitable for the Evaluation of Text Simplification. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 738–744. doi:10.18653/v1/D18-1081
 - [131] Emre Sülün, Metehan Saçakçı, and Eray Tüzün. 2024. An empirical analysis of issue templates usage in large-scale projects on github. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–28.
 - [132] Jian Sun. 2011. Why are bug reports invalid?. In *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*. IEEE, 407–410.
 - [133] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. 2025. *Announcing the Agent2Agent Protocol (A2A)*. Google for Developers Blog. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/> Accessed: 2026-01-07.
 - [134] Chakkrit Tantithamthavorn, Ahmed E Hassan, and Kenichi Matsumoto. 2018. The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Software Engineering* 46, 11 (2018), 1200–1219.
 - [135] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
 - [136] Kishor S Trivedi, Rajesh Mansharamani, Dong Seong Kim, Michael Grottko, and Manoj Nambiar. 2011. Recovery from failures due to mandelbugs in it systems. In *2011 IEEE 17th Pacific Rim International Symposium on Dependable Computing*. IEEE, 224–233.
 - [137] Erdem Tuna, Vladimir Kovalenko, and Eray Tüzün. 2022. Bug tracking process smells in practice. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 77–86.
 - [138] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi Conference on Human Factors in Computing Systems extended abstracts*. 1–7.
 - [139] Giri Vijayaraghavan and Cem Kaner. 2003. Bug taxonomies: Use them to generate better tests. *Star East* 2003 (2003), 1–40.
 - [140] Dhaval Vyas, Thomas Fritz, and David Shepherd. 2014. Bug reproduction: A collaborative practice within software maintenance activities. In *COOP 2014-Proceedings of the 11th International Conference on the Design of Cooperative Systems, 27-30 May 2014, Nice (France)*. Springer, 189–207.
 - [141] Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. 2025. Unveiling privacy risks in LLM agent memory. *arXiv preprint arXiv:2502.13172* (2025).
 - [142] Di Wang, Matthias Galster, and Miguel Morales-Trujillo. 2024. Application Monitoring for bug reproduction in web-based applications. *Journal of Systems and Software* 207 (2024), 111834.
 - [143] Di Wang, Matthias Galster, and Miguel Morales-Trujillo. 2025. Information needs in bug reports for web applications. *J. Syst. Softw.* 219, C (Jan. 2025), 21 pages. doi:10.1016/j.jss.2024.112230
 - [144] Dingbang Wang, Zhaoxu Zhang, Sidong Feng, William GJ Halfond, and Tingting Yu. 2025. An Empirical Study on Leveraging Images in Automated Bug Report Reproduction. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 27–38.
 - [145] Dingbang Wang, Yu Zhao, Sidong Feng, Zhaoxu Zhang, William GJ Halfond, Chunyang Chen, Xiaoxia Sun, Jiangfan Shi, and Tingting Yu. 2024. Feedback-driven automated whole bug report reproduction for android apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1048–1060.
 - [146] Helen J Wang, John C Platt, Yu Chen, Ruyun Zhang, and Yi-Min Wang. 2004. Automatic Misconfiguration Troubleshooting with PeerPressure.. In *OSDI*, Vol. 4. 245–257.
 - [147] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
 - [148] Shaosheng Wang, Lu Lu, Shaojian Qiu, Qingyan Tian, and Haishan Lin. 2025. DALO-APR: LLM-based automatic program repair with data augmentation and loss function optimization. *The Journal of Supercomputing* 81, 5 (2025), 640.
 - [149] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2024. Testeval: Benchmarking large language models for test case generation. *arXiv preprint arXiv:2406.04531* (2024).
 - [150] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).
 - [151] Yanlin Wang, Wanjuan Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: survey, landscape, and vision. *Automated Software Engineering* 32, 70 (2025). doi:10.1007/s10515-

- 025-00544-2
- [152] Zexin Wang, Jianhui Li, Minghua Ma, Ze Li, Yu Kang, Chaoyun Zhang, Chetan Bansal, Murali Chintalapati, Saravan Rajmohan, Qingwei Lin, Dongmei Zhang, Changhua Pei, and Gaogang Xie. 2024. Large Language Models Can Provide Accurate and Interpretable Incident Triage. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. 523–534. doi:10.1109/ISSRE62328.2024.00056
 - [153] Ming Wen, Junjie Chen, Yongqiang Tian, Rongxin Wu, Dan Hao, Shi Han, and Shing-Chi Cheung. 2019. Historical spectrum based fault localization. *IEEE Transactions on Software Engineering* 47, 11 (2019), 2348–2368.
 - [154] W Eric Wong, Hira Agrawal, and Xiangyu Zhang. 2023. Slicing-Based Techniques for Software Fault Localization. *Handbook of Software Fault Localization: Foundations and Advances* (2023), 135–200.
 - [155] Xiaoxue Wu, Wei Zheng, Minchao Pu, Jie Chen, and Dejun Mu. 2020. Invalid bug reports complicate the software aging situation. *Software Quality Journal* 28 (2020), 195–220.
 - [156] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.
 - [157] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
 - [158] Xin Xia and David Lo. 2023. Information Retrieval-Based Techniques for Software Fault Localization. *Handbook of Software Fault Localization: Foundations and Advances* (2023), 365–391.
 - [159] Xin Xia, David Lo, Ying Ding, Jafar M. Al-Kofahi, Tien N. Nguyen, and Xinyu Wang. 2017. Improving Automated Bug Triaging with Specialized Topic Model. *IEEE Trans. Softw. Eng.* 43, 3 (March 2017), 272–297. doi:10.1109/TSE.2016.2576454
 - [160] Yingfei Xiong, Arnaud Hubaux, Steven She, and Krzysztof Czarnecki. 2012. Generating range fixes for software configuration. In *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 58–68.
 - [161] Yingfei Xiong, Jie Wang, Runfa Yan, Jiachen Zhang, Shi Han, Gang Huang, and Lu Zhang. 2017. Precise condition synthesis for program repair. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 416–426.
 - [162] Chuyang Xu, Zhongxin Liu, Xiaoxue Ren, Gehao Zhang, Ming Liang, and David Lo. 2025. Flexfl: Flexible and effective fault localization with open-source large language models. *IEEE Transactions on Software Engineering* (2025).
 - [163] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817* (2024).
 - [164] Jifeng Xuan, He Jiang, Zhilei Ren, Jun Yan, and Zhongxuan Luo. 2017. Automatic Bug Triage using Semi-Supervised Text Classification. arXiv:1704.04769 [cs.SE] <https://arxiv.org/abs/1704.04769>
 - [165] Aidan ZH Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large language models for test-free fault localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
 - [166] Geunseok Yang, Tao Zhang, and Byungjeong Lee. 2014. Towards Semi-automatic Bug Triage and Severity Prediction Based on Topic Model and Multi-feature of Bug Reports. In *2014 IEEE 38th Annual Computer Software and Applications Conference*. 97–106. doi:10.1109/COMPSAC.2014.16
 - [167] Eray Yapağcı, Yavuz Alp Sencer Öztürk, and Eray Tüzün. 2025. Agents in the Sandbox: End-to-End Crash Bug Reproduction for Minecraft. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 3095–3107. doi:10.1109/ASE63991.2025.00254
 - [168] Ayberk Yaş, Cemhan Kaan Özaltan, Gökem Ayten, Fatih Kaplama, Ömercan Devran, Baykal Mehmet Uçar, and Eray Tüzün. 2025. Evaluating relink for traceability link recovery in practice. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 80–90.
 - [169] Ziyang Ye, Triet Huynh Minh Le, and M Ali Babar. 2025. LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 629–641.
 - [170] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. Survey on evaluation of llm-based agents. *arXiv preprint arXiv:2503.16416* (2025).
 - [171] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: a survey. *Software testing, verification and reliability* 22, 2 (2012), 67–120.
 - [172] Seong R Yu. 1994. A versatile development process for small to large projects using IBM CMVC. In *Proceedings of the 1994 conference of the Centre for Advanced Studies on Collaborative research*. 75.
 - [173] Kaiyuan Zhang, Zian Su, Pin-Yu Chen, Elisa Bertino, Xiangyu Zhang, and Ninghui Li. 2025. LLM Agents Should Employ Security Principles. *arXiv preprint arXiv:2505.24019* (2025).
 - [174] Tao Zhang, Jiachi Chen, He Jiang, Xiapu Luo, and Xin Xia. 2017. Bug report enrichment with application of automated fixer recommendation. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 230–240.
 - [175] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
 - [176] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software*

- Engineering* 2, ISSTA (2025), 481–503.
- [177] Yu Zhao, Kye Miller, Tingting Yu, Wei Zheng, and Minchao Pu. 2019. Automatically extracting bug reproducing steps from android bug reports. In *International Conference on Software and Systems Reuse*. Springer, 100–111.
 - [178] Yu Zhao, Ting Su, Yang Liu, Wei Zheng, Xiaoxue Wu, Ramakanth Kavuluru, William GJ Halfond, and Tingting Yu. 2022. Recdroid+: Automated end-to-end crash reproduction from bug reports for android apps. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 3 (2022), 1–33.
 - [179] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William GJ Halfond. 2019. Recdroid: automatically reproducing android application crashes from bug reports. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 128–139.
 - [180] Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Hung Huu Nguyen, Thanh Le-Cong, Junda He, Bach Le, and David Lo. 2024. Leveraging large language model for automatic patch correctness assessment. *IEEE Transactions on Software Engineering* (2024).
 - [181] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schroter, and Cathrin Weiss. 2010. What makes a good bug report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643.
 - [182] Thomas Zimmermann, Rahul Premraj, Jonathan Sillito, and Silvia Breu. 2009. Improving bug tracking systems. In *2009 31st International Conference on Software Engineering-Companion Volume*. IEEE, 247–250.